Signed in Advance: Evidentiary Sufficiency and Settlement Authority in Contracted Compute Delivery

Fée-Fée Yang

purr@psyche-o.com

October 2026

Contents

Abstract . . . 1
1. Introduction . . . 2
2. How Compute Delivery Is Reconciled Today . . . 5
3. Threat Model . . . 6
4. Related Work . . . 8
5. The Separation Principle . . . 10
6. Formal Model . . . 17
7. System Design . . . 29
8. Implementation . . . 36
9. Evaluation . . . 37
10. Limitations . . . 49
11. Open Problems . . . 52
12. Conclusion . . . 53
References . . . 54

Abstract

Contracted compute is bought in units of hundreds of millions of dollars. Disputes over availability, capacity and degradation are resolved by reconciling provider telemetry against customer logs by hand, over weeks, and no reproducible record is kept of what was decided or on what basis. Recent proposals compile contractual intent into machine-checkable obligations and settle on an automated verification verdict.

We argue that the last step confuses two questions. Verification asks whether a body of evidence supports a claim under a stated standard, and anyone holding the same inputs can recompute the answer. Acceptance asks whether the party the claim binds agrees to be bound, and only that party, or someone it authorized, can answer. A verdict answers the first question and leaves the second open, so a system that settles on the verdict records a result the counterparty is obliged to honor only where it authorized the verifier in advance, an authorization the proposals take when the obligation is bound, as policy, and this design records on the Case at review, bounded and revocable.

We present PSYCHE, which settles delivery claims for contracted compute on that separation. PSYCHE compiles a contract into a Contract Profile pinned by digest before delivery,

admits evidence through a gate that treats provenance, coverage gaps and inter-party conflict as outcomes in their own right, packages the admitted set into a signed, replayable Proof-of-Reality Capsule, and evaluates the Capsule against the Profile into one of four results, or refuses to evaluate it. The result settles nothing. A Case is accepted only on a review signed by the party it binds, or by a delegate inside a grant that party issued. What the counterparty's silence costs is signed in advance: when the Case is armed, both parties sign the review window and the expiry form, selected from an enumerated set, under which the Case expires if the window closes without a decision.

We model the system as a fold over an append-only signed log kept by an untrusted host. Acceptance soundness states that every accepted Case names the review that settled it, signed by the counterparty or by its delegate inside a valid grant. Every state a Case reaches through silence is `expired`, under a form the silent party signed in advance, and no quantity of silence produces acceptance. Supporting properties cover order invariance, terminal monotonicity, equivocation detectability and temporal binding. We describe an implementation of the contract-to-archive pipeline and evaluate it on a synthetic corpus of 1,760 Cases: replay from the bundle alone, invariant enforcement under adversarial logs, four-valued against two-valued verification, silence under settlement terms, and a worked partial-delivery dispute.

---

1. Introduction

A GPU cloud invoices a customer for 1,000 H100-hours delivered over thirty days. The customer's scheduler shows 972 hours of usable capacity, 28 hours during which memory bandwidth ran below the contracted floor, and three nodes that never matched the contracted SKU at all. Both parties have logs. Each set of logs is internally consistent. Neither party can compel the other to accept its numbers. The contract provides for service credits and specifies that credits are the sole and exclusive remedy. Six weeks later a number is agreed on a call, and nobody writes down how it was reached.

This is the ordinary case. It repeats across an asset class that has grown faster than the institutions that would normally govern it. Several neocloud providers offer no standing uptime SLA in their standard terms, and some decline to publish one at all and negotiate it per deal [1]. Where an SLA exists, the measurement period, the exclusion list, the liability cap, and the definition of availability itself can differ between the datacenter-operator contract and the operator-customer contract, so an operator may recover a fraction of monthly rent from the facility beneath it while owing credits computed against far larger service commitments above [2]. The market reconciles bilaterally, in spreadsheets.

The natural engineering response is to make the obligation machine-checkable: compile the contract into a structured object, bind execution artifacts to it, verify automatically, settle on the result. Recent work develops this program carefully [3]. We accept nearly all of it and part company at the final step.

1.1 Thesis. Verification and acceptance are different operations, and a system that settles must keep them apart.

Verification asks whether a body of evidence supports a claim, under a given standard, over a given window. The question is one of fact. Its correctness condition is soundness with

respect to the evidence admitted and the standard applied. It can be automated, audited, and recomputed by anyone holding the same inputs.

Acceptance asks whether the obligated party agrees to be bound and to pay. The question is one of authority. Its correctness condition is authorization: whether the decision was taken by someone empowered to take it, inside their limits, on the record. No quantity of evidence produces it, because acceptance is a fact about a party's will rather than about the world.

Settlement, throughout this paper, is the settlement of a delivery claim: the record of what was delivered, and what it earns under the contract, made final between the parties. A Case settles when it reaches a terminal state (§5.3), and its record changes no further; §7.7 reads what each state leaves payable, and a `disputed` Case leaves the amount to a resolution outside the record (§7.6). Payment consumes a settled record and is made elsewhere (§5.4, §7.7).

A verification verdict, however sound, creates no obligation to pay. When a counterparty declines to honor it, the system has recorded a disagreement and called it a settlement. The conflation also carries an adoption cost we take to be decisive: a finance function will not delegate acceptance authority over material spend to a third-party algorithm, and counsel will not advise it to. The party who must be persuaded to join such a network is the party asked to surrender the most.

A counterparty can also say nothing, and contracts and the law of sale already give its silence a meaning. A service credit not requested inside the claim window is lost [1, 6]. Under the Uniform Commercial Code, acceptance of goods occurs when the buyer "fails to make an effective rejection (subsection (1) of Section 2-602), but such acceptance does not occur until the buyer has had a reasonable opportunity to inspect them" [27, §2-606(1)(b)]. In both, the consequence of silence was fixed before the silence: by the contract in the first, by the law governing the contract in the second. PSYCHE carries such a term as data, and the term it carries prices silence without reading it as consent. When a Case is armed, both parties sign how long the counterparty has to review once the verifier's result, the Sigma result of §7.5, is on the record, and which of four enumerated forms the Case expires under when that window closes without a decision: nothing owed, the invoice standing, the remedy standing uncontested, or a stated cost. The same terms fix how the counterparty's own claim for a credit against the claimant is decided.

A Case therefore reaches `accepted` only on a signature of the party it binds, given at review. Silence ends it in `expired`, under a form that party signed before the Case opened. The Sigma result alone settles nothing.

The separation makes the system deployable, and it describes how durable settlement institutions have long been built. An inspection certificate, a bill of lading, a warehouse receipt, and a meter reading filed with a grid operator are evidentiary instruments produced under agreed procedure. None of them is the moment of acceptance. They constrain what that moment can reasonably contain.

1.2 A second commitment: the fabric is not trusted. PSYCHE holds the canonical delivery record. Whoever operates it can, absent countermeasures, rewrite that record, drop its tail, or present one history to the buyer and a different history to the seller. Hash chaining alone does not prevent the last of these, since a party controlling the chain head can maintain two internally consistent chains.

We therefore treat the fabric as untrusted for integrity and rely on it only for availability. In-

tegrity comes from participant signatures, a transparency-log construction over each Case's events, and inclusion and consistency proofs that participants exchange. The same construction answers the timing problem. The argument of §5.1 rests entirely on evidence being committed before its author knows which way the dispute will run, and a published log head supplies that bound without a trusted timestamp authority (§6.9). Every window the parties sign in §5.7 is measured on the same heads.

1.3 Contributions.

1. We state the separation principle and its consequence for settlement: a Case reaches `accepted` only on a signature of the party it binds, or of its delegate inside a grant (§5). The principle concerns authority, so it survives full automation of both parties, and under full automation the record of who authorized a settlement is the only place the answer can be found (§5.6).
2. We show how silence can end a Case without anyone deciding it. Both parties co-sign, when the Case is armed, the counterparty's review window, the form the Case expires under when that window closes unused, and the price list a credit against the claimant is quoted under. Every state a Case reaches through silence is `expired` under that form, and no quantity of silence produces acceptance (§5.7, §6.10).
3. We give a **threat model** in which the fabric, both parties, and the evidence sources are all untrusted, and we state the security goals and non-goals that follow (§3).
4. We give a **formal model** of the system as a fold over an append-only signed log. Its central property is acceptance soundness. Order invariance for fork-free logs, terminal monotonicity, equivocation detectability, temporal binding and silence signed in advance support it; the third and fourth hold against an untrusted fabric (§6). A table maps each formal object to the tests that hold it (§6.12). Verification-driven settlement appears as the degenerate instantiation in which the review authority is the verifier (§6.13).
5. We formalize the separation as **derivation invariants**: the one derivation of a settled state, enforced in the event model (§5.3).
6. We decompose finality into **four finalities established independently** and show that verification-driven settlement acquires its authority by substituting the first two for the third (§5.4).
7. We describe the design and implementation of **PSYCHE** (§7–§8) and report four results from evaluating it (§9). Identical admitted evidence is judged differently under two pinned standards. On a worked contract, the amount a two-valued verifier disposes of through its error handling on a Case it cannot decide is half the contracted period. Under settlement terms, silence ended every Case in `expired` under the form its arming selected, whatever the verifier had returned. And the enumerated invariants caught a defect in our own acceptance check.

§2 develops the problem domain. §3 gives the threat model and §4 surveys related work. §5 develops the principle, §6 the formal model, §7 the system design, §8 the implementation. §9 evaluates, §10 states limitations, §11 discusses open problems, §12 concludes.

2. How Compute Delivery Is Reconciled Today

2.1 An obligation that resists observation. Contracted compute is a service whose delivery is continuous and high-dimensional, and it is observed principally by parties with an interest in the observation.

A commitment for N accelerators of type T, available at rate R, over window W, is met to a degree along several axes at once: instance availability, interconnect health, thermal behavior, memory bandwidth, storage throughput, scheduler queue latency. Each axis can sit inside tolerance while the delivered value falls outside it. Contracts respond by defining an availability metric, a measurement period, an exclusion list, and a credit schedule. The metric projects a high-dimensional signal onto a scalar, and the disputes live in the projection.

Both sides instrument. The provider holds node telemetry, hypervisor logs, and fabric counters. The customer holds scheduler records, job exit codes, and application throughput. They are collected against different clocks, aggregated over different windows, and retained under different policies. Establishing that a provider-reported degradation and a customer-reported job failure describe the same event is itself a reconciliation problem, and it is usually the first one the parties hit.

2.2 The remedy structure suppresses the record. The commercial structure compounds the difficulty. Service credits are typically the sole and exclusive remedy, capped at a percentage of fees for the affected period, and available only on a documented request filed inside a claim window [1, 6]. Three consequences follow.

The burden of proof falls on the party with the weaker instrumentation. A customer must document a shortfall in the provider's service using data the customer gathered from outside the provider's infrastructure.

The cap makes rigor irrational. When the maximum recoverable credit is a small fraction of monthly spend and constructing the claim costs engineering weeks, most shortfalls go unclaimed. The absence of disputes in this market is therefore evidence that disputing does not pay.

Because credits are exclusive, disagreements never reach a forum that would generate precedent. They are resolved privately, on confidential terms. Each dispute is resolved as though it were the first one, and the knowledge generated in resolving it leaves with the people who were on the call.

2.3 The mismatch propagates. The supply chain is layered. A facility operator sells to a neocloud, which may sell through a broker or marketplace, which sells to a buyer. Each layer negotiates its SLA independently, and the definitions need not align across layers [2]. Measurement periods differ. Exclusion lists differ. Liability caps differ. Availability itself is defined differently at each interface, so an outage that counts at one layer may not count at the next.

The obligations therefore do not net. An operator can owe customer-facing credits computed on service contract value while recovering a percentage of monthly rent from below, and no participant holds a record in which the two can be placed side by side.

High value, high volume, many layers, independent bilateral reconciliation, incompatible definitions, no shared record. Markets with that profile have built shared records before: the bill of lading, the warehouse receipt, the meter reading filed with a grid operator. Contracted

compute has none.

2.4 What the record lacks. We found no adjudicated delivery dispute in contracted compute. The nearest usable records come from the hosting market that preceded it, and they reached the public record through insolvency rather than through judgment: a customer's motion over deployed capacity and power cost pass-through [25], and a claim over unpaid energy credits and metered outages [26]. Each was resolved by transfer of the asset, with no ruling on the arithmetic. Four things are absent.

A pinned standard. The contract's availability definition lives in prose, is read differently by each side, and is not bound to the measurement at the time the measurement is taken.

Contemporaneous commitment. Evidence is assembled after a dispute arises, out of logs retained for operational reasons, at aggregation levels chosen for operations rather than for proof.

An account of what was not observed. Gaps carry most disputes, and current practice offers no object for the statement *we hold no data for this interval, and here is why.*

A reproducible derivation. The final number comes out of a negotiation. It cannot be recomputed, cannot be audited, and cannot be compared against the number produced by any other dispute.

None of the four requires a disinterested party to watch the delivery. They require each party's own observations to be committed against a shared standard before anyone knows there will be a dispute. §7 is built on that requirement.

---

3. Threat Model

The system's purpose is to produce a record two adversarial parties will both rely on. We therefore assume no participant is honest, including the participant that runs the system.

3.1 Participants and trust assumptions.

Participant	Trusted for	Not trusted for
Operator (claimant, Capsule producer)	nothing	evidence completeness, claim accuracy, timing of its own records
Customer (counterparty, review signer)	nothing	evidence completeness, good-faith review
Evidence source	nothing beyond its declared class	independence claims, coverage claims
Standards Registry	integrity of published standards and Profile schemas	commercial terms, which stay with the quoter
Fabric (the PSYCHE deployment)	**availability, ordering service**	**integrity, single-history presentation, non-censorship, the time on its heads**
Independent verifier	nothing	correctness, which is recomputable by anyone
Quoter (named in the credit terms)	nothing	the price it attests, which either party recomputes from the pinned terms
Participant registry	binding each key, for the times it is valid, to a participant, a role and a class	—

Everything else in the model depends on the fabric's row. We rely on the fabric to accept events, assign sequence, and serve reads. We do not rely on it to be honest about what it

accepted, or about when. A deployment run by one of the parties is therefore admissible for integrity, and §9.5 argues that the two parties' own evidence, on a deployment one of them hosts, is a working configuration. Such a host can still censor the other party's events, the residue §6.6 states. Every fold also reads the participant registry, which binds keys to participants, roles and classes, and nothing on the log commits to it (§10.10).

3.2 Adversary capabilities.

	Capability
A1	**Withholding.** A party declines to submit evidence in its possession.
A2	**Selective disclosure.** A party submits only the intervals favorable to it.
A3	**Backdating.** A party signs a record asserting a time earlier than the actual time of creation.
A4	**Post-hoc revision.** A party alters a record after signing it.
A5	**Equivocation.** The fabric presents log L to one participant and a divergent log L' to another.
A6	**Truncation.** The fabric drops a suffix of the log, or censors an event, and serves the result as complete.
A7	**Authority overreach.** A participant signs a review outside the scope of its grant, or after revocation.
A8	**Replay.** A participant resubmits a previously signed event to obtain a second effect.
A9	**Collusion.** Two participants coordinate against a third, including the fabric colluding with a counterparty.
A10	**Coordinated fabrication.** Both parties contemporaneously sign, in the pre-agreed form, records of an event that did not occur.
A11	**Stalling.** A counterparty neither accepts nor rejects, and holds the Case away from an outcome.
A12	**Inflation.** A party whose credit Claim its counterparty leaves uncontested asks for more than the contract prices.
A13	**Participant equivocation.** A participant signs two decisions on one Case, each naming the same predecessor.

We assume signature forgery and hash collision are infeasible, and we do not model key compromise. A10 is out of scope and we say why in §3.4.

3.3 Security goals.

G1 Acceptance authenticity. A derived `accepted` names the signature that produced it: a counterparty-signed accept, or a delegate's inside a valid grant, with quorum satisfied. Formalized as Theorem 1, relative to the participant registry (§10.10).

G2 Non-equivocation. A fabric that presents divergent histories to two participants can be proven to have done so once they hold two of its heads, or two inclusion proofs, in one of the conflicting forms Theorem 4 lists; a missing consistency proof proves nothing (§6.6). A participant that signs two decisions on one predecessor is reported with the pair (§6.2).

G3 Evidence integrity. Alteration of admitted evidence after packaging is detectable by any holder of the Capsule. Addressed by the digest chain (§7.4).

G4 Temporal binding. Evidence admitted to a Case is provably committed before a published bound, without reliance on the signer's asserted clock. Theorem 5. This defeats A3.

G5 Coverage honesty. An unobserved interval is counted as unobserved, in both directions. Addressed by the BoundaryRecord and the coverage gate (§7.3).

G6 Authority containment. A decision exceeding its grant is detectable from the log alone, without consulting the granting party. Addressed by grant binding in the review event (§7.6).

G7 Recomputability. Any party holding the ReplayBundle derives the same result as the originating evaluation. Definition 1 and §9.1.

G8 Silence signed in advance. Silence ends a Case in `expired`, under the form both parties signed at arming; a credit Claim left uncontested under EX-03 stands at the quote the named quoter attests under the pinned price list. Every window the terms state closes on a published head. Theorem 6. This defeats A11 and A12.

3.4 Non-goals. Ground truth. The system establishes what evidence was committed, under which standard it was evaluated, and who accepted the result. Whether delivery occurred lies outside it, and §5.3 argues that a system claiming more would be claiming what it cannot discharge.

Coordinated fabrication (A10). Two parties who both sign a false record contemporaneously defeat the design. Contemporaneous commitment raises the cost, since each must commit before learning which fabrication serves it, and non-repudiation raises the consequence once discovered. Neither eliminates the attack. We doubt any system at this layer can, and a design that claimed otherwise would be relying on an unstated assumption about at least one party's honesty.

Availability under fabric denial of service. A fabric that refuses all writes, or stops publishing heads, halts every Case and every window. Liveness of that kind is an operational concern rather than a protocol one. A fabric can also decline to include a particular event, and the construction leaves no proof that it did (§6.6). A window that waits on a record the network's verifier or quoter never makes does not open, and the Case ends at the deadline the parties armed it with (§5.7); a network component that withholds its record is the same operational concern.

Confidentiality of evaluable structure from admitted verifiers. Verifiers necessarily learn the thresholds they evaluate against. §7.2 gives the disclosure boundary and states what remains open.

Compelling acceptance. A counterparty may refuse, and the design keeps that power with it: its silence ends a Case in `expired`, at the cost it signed when the Case was armed, and in no other state. §1.1 explains why a system that could compel acceptance would not be one anybody joined.

---

4. Related Work

Verification-native clearing. RAILS [3] develops a clearinghouse protocol for agentic commerce over seven primitives, among them an Obligation Object compiling natural-language intent into a signed machine-clearable contract, an Evidence Envelope binding execution artifacts under explicit admissibility classes, and a Verification Mesh applying several verifier types under conflict-resolution and escalation rules. The two papers address different problems with overlapping machinery: RAILS is concerned with a principal and its agent, we with the two sides of a contract. We read the independent convergence on compiled obli-

gations, bound evidence and layered verification as evidence that the problem admits few other decompositions.

The difference is who holds review authority. Where the mesh cannot reach its confidence threshold, RAILS escalates to a human arbiter, and *the arbiter is itself a verifier in role-tag terms, its output re-entering adjudication as one more verifier vote* under the same aggregation rules as any machine verifier [3, §6.3]. There the obligated party submits evidence to the party that decides. In PSYCHE the obligated party is the only party that decides, and a Case it leaves undecided expires under terms it signed. §6.13 states the relationship in the model's terms: the two designs differ in the value of one parameter.

Verify-then-pay. TessPay [19] escrows funds at delegation, accumulates a Proof of Task Execution during execution, and releases the escrow when that proof satisfies the obligation's verification predicates. The predicate itself moves the money. No party accepts.

Verifiable SLAs under proof. Castillo et al. [20] compile SLA clauses into verifiable predicates, monitor them inside trusted execution environments, organise timestamped telemetry into Merkle trees, and aggregate service-level indicators under zero-knowledge proofs. The output is a violation claim an arbitrator or insurer can check without access to the underlying data. Of the three this is nearest to ours in subject matter, and the trust properties it establishes over its inputs are the ones an evidence gate wants. Its proof settles nothing by itself: the arbitrator or insurer who checks it decides, so it sits beside the separation of §5 rather than against it (§6.13). It requires trusted hardware at the point of measurement, which §5.1 argues is unnecessary: contemporaneous commitment by both parties to a pinned standard suffices to make self-interested records bind.

Our objection to the first two is aimed at the destination and at none of the constructions. An Evidence Gate would admit each of the three artifacts at a high strength class, and §5.3 argues that a Case carrying them still halts at a Sigma result. The same collapse appears in two designs for agentic commerce built with care, so we treat it as a tendency of the field rather than a lapse of either.

Tamper-evident logging and transparency. Our non-equivocation construction follows Certificate Transparency [7], which appends to a Merkle tree [16], publishes signed tree heads, and supplies inclusion and consistency proofs so that a log presenting divergent views can be caught by clients who compare heads. Crosby and Wallach's history trees [8] give the data structure for tamper-evident logs with efficient membership and incremental proofs, and their threat model — an untrusted logger, auditors who verify — is the one we adopt. Haeberlen et al.'s PeerReview [9] establishes accountability in distributed systems by making every node's actions verifiable against a signed log, and its notion of exposing a faulty node through conflicting signed statements underlies Theorem 4 and the fork report of §6.2. Haber and Stornetta's timestamping construction [12] is the ancestor of our temporal binding result, and RFC 3161 [13] is the alternative we avoid, since a timestamp authority reintroduces a trusted party we have otherwise removed.

Fair exchange and non-repudiation. The problem of two mutually distrustful parties exchanging items with a guarantee for both has a long literature. Pagnia and Gärtner [11] establish that fair exchange is impossible without a trusted third party, and optimistic protocols [10] respond by involving the third party only when a party misbehaves. Our setting is a variant: the item exchanged on one side is a delivery and on the other a payment, the delivery is not atomically transferable, and the third party is present but untrusted for in-

tegrity. We do not solve fair exchange. We narrow the residue to a signature by an identified party: the review that accepts a Case, and the terms, signed before it opened, that fix what its expiry costs. No third party stands where the optimistic protocol's does; a dispute leaves the record as `disputed` and is resolved outside it (§7.6).

Optimistic dispute windows. Arbitrum [28] accepts a manager's assertion about a contract's execution when nobody challenges it in time: "If no challenge occurs during the interval, then the assertion is accepted" [28]. A challenge is resolved by bisection down to a one-step proof the verifier checks. The expiry form of §5.7 has the same clock and a different consequence. Both parties sign, before the Case opens, what the close of the window costs, and the state the close produces is `expired` under that form; the assertion is accepted only when the party it binds signs. In Arbitrum the protocol's verification decides the challenge, which places it with the designs of §6.13.

Runtime verification and attested execution. A substantial literature addresses whether a computation ran as specified, through trusted execution environments, remote attestation, and runtime verification of governed agent actions [4], as well as server-side verification of tool-call claims independent of model-generated rationales [5]. This work sits beneath ours and complements it. PSYCHE treats attestation output as evidence entering an admissibility gate and does not ask which proof system produced it. Attestation can establish that a computation occurred with certain properties. Whether those properties satisfy a commercial obligation is a question about a contract, and no attestation answers it.

Authenticated data feeds. Where evidence originates outside the parties, the question becomes how an external observation is bound to a claim without trusting whoever relayed it. Town Crier [17] answers it with a trusted execution environment that fetches data and signs an attestation an on-chain contract can check, and DECO [18] answers it without special hardware: a prover convinces a verifier that a TLS session returned a particular value. Our spot-check transcript occupies the same position — the network's oracle, which we call Sphinx, issues the spot checks — and carries a narrower obligation: it establishes that a spot check was issued, answered, and bound to the head of Sphinx's issuance log, and it makes no claim about the truth of the answer. We weaken it deliberately, for the reason §5.3 gives about Sigma.

Replicated state machines. The fold of §6 is a deterministic state machine over an ordered log, the object that replicated state machine designs replicate [14]. We do not replicate for fault tolerance. Participants derive state independently from a log they can each verify, which yields agreement without consensus among them, at the cost of relying on the fabric for ordering and detecting rather than preventing its misbehavior.

Provenance. Our evidence model records producer, method, and time in a manner close to the PROV data model [15], and a Capsule can be read as a provenance graph over a claim. We depart in requiring provenance assertions to be signed by the asserting party and admitted through a gate that can reject them.

---

5. The Separation Principle

5.1 Why self-interested records can bind. The primary evidence in a delivery dispute is produced by interested parties: the provider's telemetry and the customer's scheduler. Com-

merce has paid against interested records for centuries.

A bill of lading is issued by the carrier. The carrier is party to the contract of carriage and has an evident stake in what the document says. Banks release payment against it anyway, under letters of credit, in transactions where no disinterested party stood at the quayside and counted.

The bank has checked nothing about the cargo. It examines the document on its face and disclaims responsibility for its genuineness and accuracy [22]. The document is worth paying against because a false bill costs the carrier more than a true one. Statute fixes what the carrier must state and lets it decline what it has no reasonable means of checking, which is why bills read *said to contain* [23, Art. III rr. 3, 5]. Once stated, the statement stops being the carrier's to revise: prima facie evidence between carrier and shipper, and proof to the contrary inadmissible once the bill reaches a third party acting in good faith [23, Art. III r. 4]. Where a carrier signs clean against a letter of indemnity knowing better, it answers to whoever relied on the bill, and the indemnity, given for a fraudulent purpose, protects it from nothing [24].

A carrier usually knows the present facts at loading; *apparent order and condition* means that. It is bound because a statement, once relied on, can no longer be withdrawn.

Three properties do the work.

The record is **contemporaneous**, made at the time of the event and before any dispute exists.

Its **form is fixed in advance**, by the parties or by convention, so what is recorded is settled before the issuer's interest is known.

It is **signed, and alteration is detectable**, so the record stands as signed once the issuer's interest becomes clear.

These buy a weaker guarantee than trust. No participant is asked to believe the carrier honest, only that it commit early, in a shape agreed beforehand, to something that stands as signed. Honesty is expensive to establish and impossible to verify. Commitment under uncertainty is cheap, checkable, and sufficient.

The argument depends on *when* the record was made, and a self-interested party can assert any time it likes. §6.9 discharges this: a record included in a log whose head was published at time T provably existed before T, and the bound holds regardless of what the signer's clock claimed. The log that secures integrity also bounds the time.

We take this as the design premise. PSYCHE requires no disinterested observer of delivered compute hours, and we do not believe one exists today at useful coverage. Some properties of an allocation — the physical identity of the accelerator, its memory-topology class, a coarse geographic bound — are in principle establishable by the customer itself, from measurements of physical behaviour, without the provider's cooperation and without a vendor key; recent work reports such a construction [21]. Where such a primitive is available the gate admits its output at a strength class above either party's telemetry, and for two different reasons under §7.3's independence test: the identity and class claims are the customer's own measurements of a physical property the provider cannot restate, and the location claim is anchored on public network landmarks operated by neither party. Such a primitive does not reach the quantity this system settles: hours delivered within a window, exclusivity, and contracted floors remain unestablished by any such measurement. PSYCHE requires each party's own observations to be signed, committed against a standard pinned beforehand,

and bound to a published time. Where third-party evidence exists — hardware attestation, independently operated telemetry, a Sphinx spot-check transcript — the gate admits it at a higher strength class. The system's value does not rest on such evidence.

5.2 What the separation buys. Verification produces a dossier: an admitted evidence set, a pinned standard, a derivation anyone can recompute, and an explicit account of what was not observed. Acceptance is a signed act by an authorized party holding that dossier, and so is refusal, attributable and bound to the object it turns on. A counterparty that does neither is held to what it signed when the Case was armed: the Case expires under the form those terms select (§5.7). §1.1 states why no outcome can be derived from the dossier alone.

5.3 Derivation invariants. The principle is easy to state and easy to lose under delivery pressure: a settled state written directly, a timeout handler that reads silence as consent, a display field a projection later reads back as authoritative. Each such path is defensible in isolation and fatal in aggregate. We therefore state the principle as the one derivation the fold performs, and enforce it in the event model rather than in application policy.

Let a **Case** be the unit of governed settlement: one canonical identifier, one append-only signed event history, one derived state. The Case's terminal state is drawn from

```
final_state ∈ { accepted, rejected, disputed, expired, withdrawn }
```

and is derived by a fold over the signed log alone. The counterparty review decision is drawn from

```
claim_review.decision ∈ { accept, reject, dispute, abstain }
```

> **Invariant I — Canonical Acceptance.** `accepted` is derivable exactly when the fold has admitted a `claim_review` carrying `decision = accept`, signed by the counterparty or by a delegate inside a valid grant, with the settlement policy's quorum satisfied.

> **Invariant II — Derivation.** `accepted` is derived from the act of Invariant I. Beside that act the fold reads the Delivery Claim, the Proof-of-Reality Capsule, the Sigma result at any value, the VerificationReport, external evidence receipts and attestations, Intelligence output, the Remedy Quote, published heads, the closing of a window, the passage of time, payments and payout triggers, and insurance or warranty underwriting acceptance.

> **Invariant III — Claim Scope.** A Delivery Claim asserts work performed over a pinned window under a pinned standard.

> **Invariant IV — Assertion Boundary.** A Case carries exactly one Sigma result. The result concerns the relationship between an evidence set and a claim under a standard. It says nothing about ground truth and confers no settlement authority.

> **Invariant V — Fold Inertness.** Downstream commercial events — payables, credits, insurance review, archive — consume `final_state`. Funds move in the external payment

> domain. A credit against the claimant is claimed on a Case of its own (§5.7), and settling it leaves the original Case where it stood.

> **Invariant VI — Intelligence Separation.** Aggregate intelligence derived across Cases is published on its own surface; the acceptance chain reads the Sigma result, the review and the fold.

Invariant II names the one derivation, and §9.2 drives the fold over the lawful path, hostile input and delegation and reports what it derives.

Invariants I and IV divide the work between two parties. The verifier judges the evidence: whether the admitted set supports the Claim under the pinned standard. The counterparty decides acceptance: whether it is bound. When the counterparty decides nothing, the fold reads no decision into its silence. Once a published head shows the review window closed, the Case expires under the form both parties signed at arming; the head is the evidence of the time, and the form is the price of the silence.

Invariant IV gives something up. Sigma may return `claim-supported` on a Case where delivery was in fact deficient, if the admitted evidence supports the Claim under the pinned standard. No soundness defect follows. Sigma's proposition is *this evidence supports this claim under this standard*, which is checkable. The proposition *this delivery occurred as claimed* is not checkable by any party in this system, and a verifier that asserted it would be making a claim it cannot discharge and then moving money on it. The narrower assertion is the honest one. Whatever residual judgment remains belongs to the party who bears the loss, and it exercises that judgment at review.

Invariant VI exists because an intelligence layer is the most natural place for authority to leak back in. A system that computes a provider's historical evidence reliability will be asked, sooner or later, to let that score influence a Case. Under Invariant VI the score is published and consumed by parties making their own decisions, and it touches no Case.

5.4 Four finalities. A settlement record answers four questions. Answering them in four events keeps each authority with the party that holds it.

Evidence finality establishes exactly which evidence set was reviewed. It is achieved by the digest chain, the signatures, the log inclusion proofs, the Standards Registry snapshot `D(P)` that pins the Profile (§7.2), and the replay bundle. Once it holds, the question *what did you look at* has one answer that every party can recompute independently.

Measurement finality establishes what the admitted evidence says about performance against the pinned metric, and what that performance is worth under the contract. For the example in §1 this is the determination that availability and bandwidth ran below the contracted rates by stated amounts, computed from the Profile's own channel weights and tolerances, together with the credit that shortfall earns under the remedy schedule. It is a computation, and a system can establish it with high confidence and have settled nothing. §9.4 reports, for a worked delivery, which of the two halves each of four variants reaches.

Counterparty finality establishes the obligated party's decision: signed at review, by the counterparty or by a delegate inside its limit. Of the four, this alone produces `accepted`. Where the counterparty gives no decision, the Case reaches no counterparty finality; it ex-

pires, and what expiry costs was signed in advance (§5.7).

Commercial finality establishes how money, credit, coverage, and archive move. It consumes the terminal state.

The pairwise conflations each break something specific. Running evidence finality together with measurement finality hides the standard: the parties agree on what was observed while disagreeing about what it means, and the disagreement surfaces later as a dispute about arithmetic. Running measurement finality together with counterparty finality is the failure this paper is about, and it manufactures consent from a computation. Running counterparty finality together with commercial finality couples the record to the payment rail, so that a settled Case cannot be recorded until money is ready to move, and a system that holds funds acquires an interest in its own adjudications.

A verification-driven settlement system establishes evidence finality and measurement finality, then treats their conjunction as counterparty finality. Everything it says about the world may be correct. It has skipped the only one of the four that carries authority given on the Case. The expiry form keeps the four apart when nobody decides: the measurement is the verifier's, the Case expires without a decision, and what that costs is a term the counterparty signed before the measurement existed.

5.5 What the separation costs. The separation returns one decision to an authorized party. Everything upstream of it — admissibility, gap detection, conflict detection, coverage checking, quantity computation, application of the standard — is automated and reproducible, and the six weeks of §1 are spent there. Given a complete and recomputable dossier, the decision is one signed review, and §10.4 states that we have not measured how long it takes.

The decision can be delegated to an agent, and we expect delegation to become the common case. §5.6 shows that delegation sharpens the separation.

The counterparty can also decline to decide. §5.7 gives its silence the cost both parties signed in advance, and the state `expired`. A counterparty that signed no such terms leaves the Case to expire at its deadline, under no form.

5.6 Acceptance under full automation. Compute is contracted by people today. It will be contracted by agents, on both sides, and an agent that accepts is software reaching a conclusion from evidence. The separation has to hold when both sides are software, so it cannot rest on who staffs the decision. Consider three arrangements, the second and third automated to precisely the same degree.

A human buyer accepts. The authority is the buyer's.

An agent accepts under a grant from the buyer. The authority is still the buyer's, exercised through a delegate whose scope is explicit, bounded, and revocable. If the agent accepts something it should not have, the buyer is bound and bears the loss, because the buyer conferred the power to bind it. The buyer can narrow the grant, revoke it, or keep a class of Cases for its own review.

A network-operated verifier settles. The authority is the network's, or the buyer's once, by a clause signed before the window. If the verdict is wrong, the buyer is bound by a decision it never signed, under an authority it cannot withdraw from the Case.

The second and third differ in one respect that survives the buyer's advance signature: who can take the power away, and when. Under a grant the buyer narrows or revokes it on any

Case; under a clause signed before the window the buyer bears the consequence of a wrong verdict just the same, and cannot withdraw the authority from a Case once it runs. Both are software, and both can name the principal; only the grant names it on the Case, with the scope it conferred.

An agent acting inside its authority binds the principal; an agent acting outside it does not. Automation leaves that rule standing and moves every dispute onto the boundary of the authority.

The agent-to-agent case therefore strengthens the argument. When a person reviews, the fact that a person reviewed is visible; the separation is real but carries no weight, because nobody needs the record to tell them who decided. When both sides are agents, *on whose authority did this settle* becomes the only question that determines liability, and it becomes invisible. It has to be in the record: a grant, a signature attributable to a party, a scope a reader can check. A design that cannot produce that record has lost track of acceptance.

One version of this defeats us. If buyers grant their agents unlimited authority to accept whatever the evaluation supports, the outcome is indistinguishable in practice from settlement driven by the evaluation. The legal difference survives — the buyer chose the arrangement and can be shown to have chosen it — and the commercial difference does not.

We expect this to be the equilibrium for one segment and not the other. Where a delivery period is small and settles frequently, no finance function will review it, and wide grants will be ordinary; the separation is legally real there and commercially invisible. Where a period is large, settles monthly, and is the collateral for a facility, the acceptance is a board-visible act and the separation makes the record usable. Our commercial argument is a claim about the second segment. The formal results hold across both, since they describe structure rather than preference.

The design denies that choice its quietness. The grant constitution of §6.1 sits inside the Profile recipe, one of the four artifacts `D(P)` commits to, so the ceiling travels with the standard and a counterparty who checked one has checked both. A machine holder may accept only against the Sigma results the constitution admits, `claim-supported` alone unless the Profile widens it, and whether a holder is a machine is read from the registry rather than from the grant that confers the authority. A buyer who wants the collapse must write it down in advance, in an artifact the counterparty reads before agreeing to be measured. The counterparty can decline. The arrangement is reached in the Profile, read by the counterparty before the window.

§5.1 makes the same move for evidence. A record binds because it was committed before the outcome was known. So does a limit on what may be delegated, and so does the price of silence.

§11 states what remains open.

5.7 Silence, priced in advance. Silence is the cheapest way to stall a Case (A11). A counterparty that never reviews holds the claimant away from any outcome, and a claimant that never answers a credit Claim does the same to the counterparty. An expiry alone ends the wait and prices nothing: a provider that delivered in full receives `expired`, the same state as one that delivered nothing, and what that state costs each party is left to a conversation.

The law of sale answers the buyer's version of the problem with a consequence fixed before the silence: acceptance of goods occurs when the buyer "fails to make an effective rejection

(subsection (1) of Section 2-602), but such acceptance does not occur until the buyer has had a reasonable opportunity to inspect them" [27, §2-606(1)(b)]. The rule has two parts, an opportunity to inspect and a consequence for silence after it. The Code's consequence is acceptance. PSYCHE keeps the two parts and changes the consequence: the Case expires, and what expiry costs the silent party is selected from the enumerated set the settlement standard states [29, I.1.1, Annex 2.4]. PSYCHE does not rely on the Code, which governs goods. The parties write both parts into their own terms, and the Case carries them.

The terms. When a Case is armed, one party signs the arming event and the other co-signs the same content. Besides the standard `D(P)`, the Case's review deadline and its value at risk, the arming carries the Case's settlement terms:

- the **review window**, counted from the earliest readable head that includes the Sigma result, so the counterparty's opportunity to inspect opens when the Sigma result is on the record;
- the **expiry form**, one of four: EX-01, nothing is owed on the claim either way; EX-02, the invoice stands and the remedy lapses; EX-03, the remedy stands uncontested, the silent party having spent its opportunity to contest it; EX-04, a stated cost, fixed in advance and independent of the merits;
- optionally, **credit terms**: the review window and expiry form for a credit Claim the counterparty brings against the claimant, the date after which no such Claim may be made, that Case's deadline, a quoter, and the digest of the commercial terms a credit is priced under.

The terms carry these fields. An arming whose settlement terms carry a field outside them arms nothing, and nothing built on it stands.

Expiry. When the review window has closed and the counterparty has signed nothing, closing the window ends the Case in `expired`, and the fold records the form it expired under. The form is read from the terms alone: a supported Claim, an unsupported one and an inconclusive one expire under the same form, chosen before any of them existed. What the form costs is read outside the fold, from the terms, and moves money only with the payer's own authorization (§7.7). A Case armed without settlement terms expires at its deadline with its form unstated.

Decisions. The counterparty may decide at any point before the window closes. An accept settles the Case; a reject and a dispute are terminal at once, and a `disputed` Case leaves the record for resolution outside it (§7.6). A decision after the window closed arrives on an expired Case and is ignored.

Credit Claims. A shortfall is claimed on a Case of its own, `c/credit`, and each Case admits one. The original counterparty arms it by presenting the original Case's signed arming, which the original claimant signed or co-signed; the credit arming itself carries only the credit claimant's signature. The credit Claim must be made before the date the credit terms state, as the earliest head that includes it shows. The Case then runs in the other direction. The credit claimant files a Capsule, and the verifier evaluates it under the original standard: the credit Claim asserts a shortfall, and its Sigma result says whether the admitted evidence supports it. The original claimant reviews, and its silence leaves the credit Case to expire under the credit expiry form. Where the terms name a quoter, the quoter's attested price for this Case's Capsule, computed under the commercial terms whose digest the arming pinned, is recorded on the Case once the Sigma result it prices is on the record; a quote priced under

any other terms, or attested by anyone else, is not a quote on this Case. On such a Case the review window is counted from the earliest readable head that includes both the Sigma result and the quote, so the original claimant has seen the price before its window runs, and every head a closing may reference commits to the quote; a credit Case whose terms call for a quote that is never recorded opens no review window and ends at its deadline, with nothing standing. A quote stands only if the earliest readable head that includes it leaves a full review window before the credit Case's deadline; a later quote is not a quote on this Case, so a timeout at the deadline leaves nothing standing whichever of the quote and the timeout the fabric folded first, and silence never lets a quote stand on less than the window the parties signed. Under EX-03 what stands uncontested is that quote and nothing larger. An uncontested credit pays what the contract prices, and either party can recompute the price from the terms it signed.

Closing a window. A window closes on a published head. The fabric or either party may announce the close with an event that references a head at or after it, and the close is computed from the Case — its terms and the heads that date the events that open the window — and read from the Case alone. The head is the evidence that the window closed. A fabric that withholds the announcement no longer holds the Case away from the expiry the parties priced, since either party can make it, unless the fabric also censors that party's event (§6.6).

§6.10 states what this buys as a theorem. §9.2 tests each route by which silence, a closing or a quote could come to produce a decision, and §9.3 settles the whole corpus under settlement terms with nobody reviewing.

---

6. Formal Model

The model below formalizes the separation of §5 and defines the properties that §7 implements and §9 evaluates. We model a Case as a fold over an append-only signed log. The model is deliberately small. It says nothing about evidence semantics, which vary by Profile, and everything about who can move a Case to a settled state.

One property carries the contribution. **Theorem 1, acceptance soundness**, is the formal content of the separation principle: every accepted Case names the review that settled it, signed by the counterparty or by a delegate inside a grant the counterparty issued. The rest are supporting properties, and each exists because Theorem 1 would be unusable without it. Determinism (Theorem 2a) makes a log's derived state unique, and order invariance (Theorem 2b) makes it independent of how the fabric ordered events the Case does not relate, so two parties reading the same events agree without coordinating. Terminal monotonicity (Theorem 3) keeps a settled Case settled. Equivocation detectability (Theorem 4) and temporal binding (Theorem 5) extend the guarantee from a log to a *deployment*, where the fabric that keeps the log is one more party nobody has agreed to trust, and Corollary 5.1 carries that binding to the standard the evidence is read against. Theorem 6 fixes what silence can reach: `expired`, under a form signed in advance. A reader interested only in the separation may read §6.3 and §6.10 and skip to §7.

6.1 Objects. Let **R** be the participant registry and **P** the set of participants it names, each bound by `R` to a verification key, a role and a class. `R` is versioned: every key carries the time from which it is valid and, once retired, the time until which it was, rotation registers the

successor before it retires the predecessor, and `key(p, t)` is the key `R` holds valid for `p` at time `t`. A **grant** is a tuple

```
g = (issuer, holder, from, until, max_value, allowed_outputs)     issuer, holder ∈ P
```

conferring review authority over the Case on `holder`, valid over `[from, until)` until the issuer revokes it. A grant is an event on the Case's own log, so it confers authority over that Case and no other. `allowed_outputs` names the Sigma results the holder may accept against and `max_value` the largest value a review under the grant may dispose of. Whether the holder is a machine is read from the holder's participant class in the registry, so the issuing party does not describe its own delegate. Grants and revocations are events, so the grant state at any point is derived by the same fold that derives Case state.

Grants are bounded by the Profile. A **grant constitution** `K` is a component of the Profile recipe, and therefore one of the four artifacts `D(P)` of §7.2 commits to. It states the limits within which any grant on a Case under that Profile may be issued:

```
K = (issuers, holders, machine_outputs, max_delegable)
```

`issuers` names the roles that may confer review authority; the implementation admits the obligated party alone. `holders` names the roles that may hold it, a party alone unless the Profile adds others, so that §6.13's degenerate instantiation is reached in the Profile, where the counterparty reads it. `max_delegable` caps the value a single delegated review may dispose of. `machine_outputs` is the set of Sigma results a non-human holder may accept against, `{claim-supported}` alone unless the Profile widens it.

A grant issuance is admissible only when the grant it confers lies within `K`: the issuer's role is in `issuers`, the holder's is in `holders`, a machine holder's admitted results are a subset of `machine_outputs`, and the grant's ceiling does not exceed `max_delegable`. A Profile that arms no constitution authorizes no delegation: absence is refusal rather than permission, since a standard silent on delegation has not conferred any.

Because `K` sits inside a hashed artifact, `D(P)` and the constitution stand or fall together: two Cases asserting the same standard permit their delegates exactly the same things, and a counterparty who checked the commitment has checked the ceiling. Carrying `K` in the arming event's own payload would place it beside the commitment instead of inside it, and the fold refuses an arming that does so. `K` is therefore fixed before the window by Corollary 5.1. A deployment that wishes to widen it may do so, in the Profile, where the counterparty reads it before agreeing to the standard and can decline. Widening it takes a new commitment.

The **arming** event opens a Case. It is signed by one party and co-signed by the other over the same content together with the Case reference, so a co-signature given for one Case cannot be carried into another (a credit Claim's arming carries an original arming instead, §5.7), and it carries `D(P)`, the claimant, the counterparty, the Case's review deadline and value at risk, and optionally the settlement terms `T` of §5.7. Whatever the arming fixes, no later event moves.

An **event** is

```
e = (id, case, kind, payload, signer, prior, σ)
```

where `σ` is `signer`'s signature over the canonicalized remainder. **Authority events** — arming, the Claim, grant issuance and revocation, reviews, window closings and terminal events — carry in `prior` the digest of the authority event before them on the Case, and the first carries none. Evidence and record events — the Capsule, the Sigma result, a quote, reports and

payments — carry no `prior`; an event that relies on one names it by digest. A **log** `L = ⟨e_1 … e_n⟩` is the sequence of a Case's events as the fabric ordered them and committed to them under its signed heads (§6.6). Write `L ⊑ L'` when `L` is a prefix of `L'`.

The **authority chain** of `L` is the sequence of admitted authority events, each naming its predecessor. Two authority events naming the same predecessor are a **fork**.

A **causal order** $\prec$ is a partial order on events. `e ≺ e'` holds when `e'` names `e` by digest — as its `prior`, or as the Capsule or Sigma result it relies on — when `e'` references a published head that commits to `e`, or when the Case's transition rules make `e'` reachable only after `e`, as the Capsule is reachable only after the Claim.

The **state space** `S` contains a distinguished non-terminal region and the terminal states of §5.3. The **fold** is

$$F : L \to S, \qquad F(L) = \delta^*(s_0, L)$$

for a transition function $\delta : S \times \text{Event} \to S$ that is total and deterministic. The fold also reads the fabric's signed heads `H` over `L`, the observers' receipts ρ for them (§6.6, §6.8) and the registry `R`, so `F` is a function of `(L, H, ρ, R)`; we write `F(L)` where the heads, receipts and registry are fixed.

6.2 Admissibility of events to the fold. δ ignores an event when its signature does not verify under `key(e.signer, sequenced(e))`, the key `R` holds valid for the signer at the earliest readable head that includes the event; when the signer lacks standing for the event kind on that Case; when it is a review that names a Claim or Capsule other than the Case's; when it is an arming that lacks what §5.7 requires of it, the other party's co-signature over the Case reference and the arming's content, verifying under that party's key, or for a credit Case the signed original arming it is brought under; when the event repeats one already folded; when it is an authority event whose `prior` is not the head of the chain; when it would give the Case a second Claim, Capsule, Sigma result or quote; when it is a quote recorded before the Case's Sigma result, or whose earliest readable head leaves less than the review window before the Case's deadline (§5.7); when it is a timeout, withdrawal or window closing that omits the Sphinx log anchor the Case's Capsule declared; or when the Case is already terminal.

An event acting under a grant is judged in the grant state the chain establishes at its position. A revocation earlier in the chain bears on it, and a revocation later in the chain leaves it standing. The review names its predecessor, and so the grant state it is judged in is fixed by what it signed.

The alternative — evaluating validity against whatever prefix the fabric happens to have folded at the point of use — makes the outcome a function of the fabric's ordering choices between two events the causal order does not relate. Theorem 2b exists to exclude exactly that dependence, and §6.9 removes it for asserted time by the same device and for the same reason: a participant may rely on a published commitment, and not on the moment a component of the system chose to act.

Of two authority events naming the same predecessor, the first admissible one the fabric sequenced extends the chain, and a later event naming the same predecessor is refused and reported, with its signer, as a fork. When one participant signed both, the pair is a transferable proof that it signed two decisions on one predecessor (A13). When two participants signed them, the fork is a race the fabric's signed sequence decided, and the log shows who signed the event that lost.

Ignoring is total: an inadmissible event contributes nothing. A7 and A8 are inert.

6.3 Acceptance — the central property. Let `ReviewAuthority(c) ⊆ P` be the participants holding review standing on Case `c`: the obligated party, and any participant acting under a grant from it. Membership is a standing that must be held. Defining it as the complement of the claimant admits the network's own verifier, and §9.2 reports what happens when it is written that way. The distinction is the whole of §6.13.

A review's admissibility collects into a predicate with the shape and discipline of the Evidence Gate of §7.3: a pure function of the log up to the review's position, whose failure leaves the Case where it stood.

```
AdmitReview(e, L, c)  ≡  verify(e.σ, key(e.signer, sequenced(e)))
                       ∧ e extends the authority chain of L
                       ∧ e names claim(c) and capsule(c)
                       ∧ (  e.signer = counterparty(c)
                         ∨  ( g := grant(e), g live in the chain before e
                            ∧ g.holder = e.signer
                            ∧ head(e).time ≥ g.from
                            ∧ sequenced(e) < g.until
                            ∧ value(c) ≤ g.max_value
                            ∧ ( e.decision = accept ∧ machine(e.signer) ⇒
                                  SigmaOutput(c) ∈ g.allowed_outputs ) ) )
```

Four of these terms name where their value comes from, because for each the obvious source is a party with an interest in the answer. `head(e)` is the time of the published head the review references: the reviewer had seen it, so it acted after it. `sequenced(e)` is the time of the earliest readable head that includes the review: the review existed before it (Theorem 5; readable is defined in §6.8). A grant's window is judged against both and against no time the reviewer asserts. Reading only the referenced head would let a reviewer act after the window closed by naming a head from inside it, and a review naming no head fails a window that opens at a stated time, since a reviewer with no head is offering its own word. `machine(holder)` is the participant class the registry records for the holder when it is registered, people and machines signing in by different means (§8); reading it from the grant would hand `K.machine_outputs` to the party it constrains. `value(c)` is the value at risk both parties co-signed at arming. A Case armed without one fixes no value; an unarmed Case takes the value its Claim declares, which the claimant alone chooses. A grant carrying a ceiling refuses a Case that fixes no value, since a Case naming no value would otherwise clear every ceiling by omission. The machine clause bounds accepts only: a delegated reject or dispute is bounded by the grant's window and ceiling.

The settled state `accepted` has one source, the act of Invariant I:

```
Acc(L, c)  ≡  ∃ e ∈ L : e.kind = claim_review ∧ e.decision = accept
                      ∧ AdmitReview(e, L, c)
                      ∧ Quorum(c) holds over the admitted accepts
```

> **Theorem 1 (Acceptance Soundness).** For every registry `R`, log `L` and Case `c`, if `F(L) = accepted` under `R` then `Acc(L, c)` under `R`.

Proof sketch. `accepted` occurs in `S` only as the image of one transition of `δ`, the accept transition of an admitted review, which is guarded on the conjunction defining `Acc`; every other transition of `δ` maps into the non-terminal region or into a terminal state other than `accepted`. Since `F` is `δ` folded over `L` from s_0, any `L` with `F(L) = accepted` contains an event at which that transition fired, and its guard held at that point. By §6.2 the event was admissible. ■

The theorem is narrow on purpose. It says nothing about the wisdom of the acceptance, the truth of the evidence, or the fact of delivery. It says that no other object in the system manufactured consent, and that every accepted Case names the review that settled it. The review traces to the counterparty's signature: on the review itself, or on the grant its delegate acted under. The theorem is also relative to `R`: who the counterparty is, which key is its, and whether a holder is a machine are read from the registry the fold is given, so a fold under a registry that binds that key to someone else proves acceptance by someone else. §10.10 states what that costs.

Three properties follow. A review failing `AdmitReview` contributes nothing and leaves the Case in whatever non-terminal state it held. Overreach is therefore detectable from the log and inert in the fold. And the obligated party retains standing without a grant, since a grant carries authority to a delegate, and a party needs none to bind itself.

The standing branch is a disjunct: `K` bounds delegation, and the principal's standing is its own. A buyer whose own registered key is of machine class therefore accepts against any Sigma result without widening anything. That is the defeating case §5.6 concedes, and here it arrives through the registry instead of through a grant. We prefer it there. Registering the party that owes the money as a machine is a louder act than a clause in a Profile, it is done once rather than per Case, and the counterparty sees which class it is dealing with before agreeing to be measured. The machine clause makes the collapse of §5.6 visible: a buyer wanting an agent to accept whatever the evaluation supports must widen `K.machine_outputs` before the window, in a document the counterparty reads.

6.4 Order invariance — supporting. Two properties are in play here. The first is that a given log has one derived state. The second is that the state does not depend on how the fabric ordered events the Case does not relate. Only the second needs an argument.

> **Theorem 2a (Determinism).** For a fixed sequence `L`, `F(L)` is unique.

Proof. `δ` is total and deterministic and `F` is its fold from s_0. ■

§9.1 exercises Theorem 2a: replaying a bundle reproduces the derivation bit for bit. Theorem 2b is the property participants need from one another.

> **Theorem 2b (Order Invariance).** Let `L` and `L'` be two sequences over the same events, each consistent with $\prec$, folded with heads and receipts under which every event's referenced head and earliest readable head carry the same times in both; the head objects themselves differ, since each commits to its own sequence's order, and only their times enter a guard. If the log satisfies the three conditions below, `F(L) = F(L')`.
>
> **C1 (No fork).** No two authority events name the same predecessor.
>
> **C2 (One of each).** The log carries one Claim, one Capsule, one Sigma result, and at most

> one quote.
>
> **C3 (Capsule before closings).** Every timeout, withdrawal and window closing is ordered by $\prec$ with respect to the Capsule.

Proof. $\prec$ contains the authority chain, so every linearization presents the authority events in the same order, and under C1 the fold admits the same ones. The fold reads the authority events in chain order, together with the state components the evidence and record events write. It suffices that each such component a guard reads at an authority event `a` is written by an event that precedes `a` in $\prec$, since the component then has the same value at `a` in both sequences, and under C2 it is written once. We take the guards in turn.

A review. It reads the Capsule, which it names by digest, and a machine delegate's accept reads the Sigma result, which it names. Grants, revocations, the value and the deadline are written by authority events.

A window closing. It reads the time the review window opened and the form the terms select, and on a credit Case whose terms name a quoter it leaves the recorded quote standing. The window opened at the earliest readable head including the Sigma result and, on such a credit Case, the quote; the head the closing references is at or after the window's close. Head times do not decrease as the log grows (§6.6), so that head is at least as large as the ones that first included the Sigma result and the quote, commits to both, and places both before the closing in $\prec$; the quote the closing leaves standing is therefore the same in every order. The form is authority state, fixed by the arming.

A timeout or a withdrawal. It reads the deadline, its referenced head, and whether the Capsule declared spot-check issuance; by C3 the Capsule precedes it in both sequences or in neither. On a credit Case whose terms name a quoter a timeout also leaves the admitted quote standing: an admitted quote was first included by a head earlier than the deadline, and the head the timeout references is at or after the deadline, so that head commits to the quote and places it before the timeout in $\prec$; a quote whose first head is later is refused in every order (§6.2).

Reports, boundary records and payments. No guard reads them.

Time is read only from heads, and the heads and receipts give every event the same bounds in both sequences, so every time-bound guard evaluates identically. ■

The condition on heads fixes time and leaves order free. The fabric chooses where a head falls, and a later head is a later time for every event it first includes; censorship and delay are the residues of §3.4 and §6.6. Theorem 2b says the outcome is the same under any ordering, between two heads, of events the Case does not relate. Receipts enter the same way. Two parties holding different receipts under a witnessed Profile can read different heads, and a receipt that arrives later can make admissible an event the fold refused; Theorem 2b compares folds over the same heads and receipts, and §10.8 states what a late receipt leaves open.

C1, C2 and C3 are properties of a log. On a log with a fork, the fold follows the branch the fabric sequenced first and reports the fork (§6.2). A second Claim, Capsule, Sigma result or quote is ignored, so on a log that violates C2 the fold keeps the one the fabric sequenced first; both are on the log, signed, and a second Capsule from the claimant or a second Sigma result from the verifier is attributable to its signer. C3 fails when a party emits a timeout or a

withdrawal while the claimant's Capsule is in flight. A claimant that files its Capsule before the deadline avoids the race, since every head a later closing references then commits to the Capsule.

The chain also fixes revocation's scope. A revocation bears on reviews after it in the chain and on none before it. Making revocation retroactive — voiding every review under the grant regardless of position — was rejected on commercial grounds. A delegate who accepts under a live grant, and a claimant who relies on that acceptance, would both be exposed to an issuer who could unwind a settled Case by revoking afterwards. Theorem 3 says a settled Case stays settled; retroactive revocation would make that promise conditional on the issuer's later conduct.

6.5 Terminal monotonicity — supporting.

> **Theorem 3 (Terminal Monotonicity).** If `F(L)` is terminal and `L ⊑ L'`, folded with heads and receipts that read every event of `L` as before, then `F(L') = F(L)`.

Proof sketch. `δ` defines no transition out of a terminal state, and every event folded after the Case reached one is ignored (§6.2). The events of `L` are read as before, so the fold reaches the same terminal state at the same position. ■

The side condition excludes what §6.4 excludes. A head the fabric signs later over an earlier prefix, with an earlier time, can re-date an event already folded; without receipts the fabric's clock is trusted anyway (§6.8), and under a witnessed Profile its late head goes uncorroborated. A receipt that arrives late can make readable a head an event referenced, and so admit an event the fold refused; §10.8 states that residue.

A settlement record that could never be revisited would be unusable, since discovery of a shortfall after finality is a normal commercial event. A shortfall found after the Case settles is claimed on a new Case: the counterparty brings a credit Claim, armed by presenting the original arming and decided under the credit terms the original claimant co-signed (§5.7). The original Case stays settled, and the dispute runs in the open, on a log of its own.

6.6 An untrusted fabric — supporting. Theorems 1 through 3 are properties of a log. They say nothing yet about a fabric that shows different logs to different participants (A5) or drops a suffix (A6). We close this with a transparency construction.

The fabric keeps each Case's events as the leaves of a Merkle tree [16] and periodically publishes a **signed head** `STH = (log, size, root, time, σ_fabric)`. Participants obtain **inclusion proofs** for their own events and **consistency proofs** between heads they have seen, and they exchange heads with each other and with any auditor on whatever schedule the Profile requires. This is the Certificate Transparency construction [7] over the history-tree data structure [8], and its accountability property is the one PeerReview [9] formalizes. A fold given a Case's heads folds exactly the events the latest head covers and names that head in its result. It refuses to fold with heads that do not commit to the events supplied, or whose time decreases as the log grows.

> **Theorem 4 (Equivocation Detectability).** Each of the following is a proof of fabric misbehaviour that a third party can check against the fabric's registered key, trusting neither participant who produced it: (a) two heads of one log with equal size and different roots;

> (b) two heads with inclusion proofs that place different leaves at the same position; (c) two heads of which the larger carries the earlier time.

Proof sketch. One append-only tree has one root at each size, holds one leaf at each position in every head that covers the position, and grows with time. Each head carries `σ_fabric`, so a pair violating any of the three is non-repudiable. ■

A fabric that shows two participants divergent histories (A5) is caught under (a) or (b) once they compare a head of equal size, or inclusion proofs for one position. A fabric that drops a suffix it has signed (A6) cannot hide it: the head that covers the suffix proves the suffix existed, and a fold given that head refuses the shorter log. A fabric that never includes an event leaves no proof at all. A head proves what the fabric included, not what it was sent, so only the censored party knows of the censorship; it keeps its own signed event and can present it outside the log. Signed promises to include a submitted event within a stated delay, as in Certificate Transparency [7], would make censorship attributable, and the implementation has none (§11). Absence of a consistency proof between two heads of different sizes is not on the list. A third party cannot tell a fabric that cannot produce one from a fabric that was not asked, so an observer that obtains none refuses to receipt the larger head and offers no proof it does not have. The guarantee is detection, not prevention, and it holds on the conditions above. We take detection with attribution as the target, since prevention would require consensus among mutually distrustful parties and would import the cost of that machinery for a property the parties can obtain more cheaply.

Under Theorem 4 a deployment operated by one of the parties is admissible for integrity, because the other party's protection comes from signed heads and inclusion proofs rather than from the host's disinterest. Neutrality becomes a property of the construction, with censorship the exception just stated.

6.7 Closing a window. `expired` is reached without a review. It follows an event announcing that a window closed: a `window_closed` for the review window the terms fix, or a timeout for the Case's deadline. The fabric is the natural emitter, and the fabric is the party §6.6 declines to trust for integrity.

A fabric that announces a close early, or never announces it, moves a commercial outcome without forging anyone's consent: under EX-02 the invoice stands when the window closes, and under EX-03 the remedy does. Theorem 1 is untouched, and Theorem 4 does not reach it either: a premature closing, correctly logged and correctly included, is not a divergent history.

One rule takes both powers away. A closing event is admissible when the fabric or either party emits it and it references a published head whose time is at or after the window's close. The close is computed from the Case — the armed deadline, or the review window's length from the terms counted from the earliest readable head that includes the Sigma result and, on a credit Case whose terms name a quoter, the quote — and read from the Case alone. A fabric that withholds the close no longer holds the Case away from the expiry the parties priced, since either party can announce it, unless the fabric censors that party's closing as well (§6.6). A fabric wishing to close early must either wait or publish a head carrying a false time. The second is the offence §6.8 makes attributable, and it is a different offence from equivocation: a single append-only tree whose heads carry uniformly false times is internally consistent, and every consistency proof over it succeeds.

A review still in flight when a window closes races the closing, and the chain decides the race. If the review extends the chain first, the closing that follows names it and finds the Case as the review left it: settled, in which case the closing is ignored, or still open after an abstain, in which case the closing expires it. If the closing extends the chain first, the review names a stale predecessor and is refused. If both name the same predecessor they are a fork, and the fabric's signed sequence decides it (§6.2); a fabric colluding with one party can decide such a race for that party. Either way the log shows which came first and who signed the event that lost.

This closes withholding and premature emission. It does not close a fabric that stops publishing heads, which stops every window and remains the liveness failure the model concedes (§3.4).

6.8 The fabric's clock. Every commitment-before-the-fact in this design reads a time off a signed head: evidence under Theorem 5, the standard under Corollary 5.1, a grant's window under §6.3, and every window of §5.7. The head is signed by the fabric, and §3.1 declines to trust the fabric for its time.

The consequences run both ways. A fabric that postdates a head can close a window that is still open. A fabric that backdates one can make a record committed after the outcome was known appear to precede it, the failure §5.1 exists to prevent. Theorem 4(c) catches a clock that runs backwards across heads, and misses a clock that is wrong in a consistent direction.

The mitigation is a receipt. A participant or auditor that receives a head signs a receipt stating its own time of receipt. A Profile may require that a head's time be corroborated by a quorum of such receipts, each within a stated tolerance, before any guard reads it; a head that meets the Profile's requirement is **readable**. Receipts from the network's own roles — the fabric, the verifier and the quoter — do not count toward the quorum, and one observer counts once. Under such a Profile the fold reads no uncorroborated time, and it reports every receipt that contradicts its head. The tolerance is a Profile parameter, bounded below by the head publication interval and by ordinary clock skew among observers.

A receipt proves less than Theorem 4 does. Theorem 4 produces a transferable proof out of `σ_fabric` alone. A clock accusation rests on `σ_fabric` for the head and on the observers for the receipts, and a receipt is only as good as its signer. Under A9 the fabric and one counterparty can collude to misreport or withhold that party's receipts. The accusation then rests on however many honest observers remain. An auditor subscribing to heads under §7.8 is the cheapest way to add one, since it holds no other role and has nothing at stake in the Case.

A clock accusation is a claim several observers can make together and none can make alone, where equivocation is a claim one holder of two heads can make by itself. It also shares the dependency of §10.5: participants who never gossip are unprotected, and the protocol cannot tell them so. We state it as a residue distinct from Theorem 4, because only a fork produces a proof that stands on the accused party's own signature.

6.9 Temporal binding — supporting. Backdating (A3) defeats the argument of §5.1, and it cannot be addressed by inspecting the signature, since the signer chooses the asserted time. A timestamp authority [13] would solve it by reintroducing a trusted party. The transparency construction already supplies what is needed.

> **Theorem 5 (Temporal Binding).** If event `e` has an inclusion proof against a head `STH`, then `e` existed before `STH` was signed, independent of any time asserted by `e.signer`. If, in addition, `STH` is readable under a Profile requiring receipts within a tolerance `τ`, and one of the receipts that made it readable was signed by an honest observer whose clock is within `Δ` of true time, then `e` existed before `STH.time + τ + Δ`.

Proof sketch. An inclusion proof establishes that `e` is a leaf of the tree with root `STH.root`. The root commits to every leaf under collision resistance, so producing `e` after `STH` was signed would require finding a second preimage for the root. The honest observer held `STH` by a true time within `Δ` of the time its receipt states, and a receipt counts toward the quorum only when its stated time lies within `τ` of `STH.time`. ■

The first half needs no clock: it orders `e` before the head. The second reads the time off an honest observer's receipt, the mitigation of §6.8. Under a Profile that requires no receipts, Theorem 5 establishes order against the head, and `STH.time` is the fabric's word for the wall time.

The bound is one-sided and coarse: it establishes an upper bound on creation time at the resolution of the head publication interval, and says nothing about how much earlier the event was created. §5.1 needs exactly that property and no more. The bill of lading argument requires only that the record was committed before the outcome was known, and a head published at the close of the delivery window establishes that for every event beneath it. Tightening the interval tightens the argument, at a cost in publication frequency.

The same device reaches the standard. §5.1 needs that half, and Theorem 5 alone does not supply it.

> **Corollary 5.1 (Standard Binding).** Let the arming event for Profile `P` on Case `c` have an inclusion proof against a head `STH`, and let the claim window open no earlier than the bound Theorem 5 gives: `STH.time + τ + Δ` under a Profile requiring receipts, and `STH.time` where the fabric's clock is taken on its word. Then `P` was fixed before any of the delivery it governs occurred.

Proof. By Theorem 5 the arming event existed before that bound. The event carries `D(P)` of §7.2, which commits to all four Profile artifacts under collision resistance, so producing a different artifact set consistent with the same commitment would require a second preimage. The window opens at or after the bound. ■

Corollary 5.1 supplies that commitment for the standard the evidence is read against, and for the grant constitution of §6.1, which the same commitment covers. The fold recomputes the commitment from the recipe it is given and refuses a standard that does not reproduce the digest the arming carries. It reports the time of the earliest readable head that includes the arming; the log does not carry the claim window, so a reader checks the corollary's hypothesis. The settlement terms of §5.7 ride in the same arming event under both parties' signatures, so Theorem 5 binds them to the same head. The bill of lading is written in a form both parties fixed before loading; here the corollary binds the form itself.

6.10 Silence signed in advance — supporting. Let a Case `c` be armed with co-signed settlement terms `T = (W, X, credit)`: the review window `W`, the expiry form `X` selected from

the four of §5.7 with its parameter [29, Annex 2.4], and optionally the credit terms. Let `opened(c)` be the time of the earliest readable head that includes the Case's Sigma result and, on a credit Case whose terms name a quoter, its quote, and let `Silent(L, c)` hold when `L` carries no admitted accept, reject or dispute on `c`.

> **Theorem 6 (Silence Signed in Advance).** Let `c` be armed with settlement terms `T`, and let `Silent(L, c)` hold.
>
> (a) If `F(L)` is terminal then `F(L) ∈ {expired, withdrawn}`, and if `F(L) = expired` the fold names `X` as the form `c` expired under. No Sigma result, no value and no quote moves `F(L)` to `accepted` or `rejected`.
>
> (b) While `F(L)` is not terminal, once a readable head at or after `opened(c) + W` exists, the fabric or either party can close the window, and closing it yields `expired` under `X`; a timeout referencing a head at or after the Case's deadline yields `expired` under `X` likewise.
>
> (c) On a credit Case whose credit terms name a quoter, the quote the result names is the one the quoter attested for this Case's Capsule under the pinned terms, and no other; under EX-03 that quote is the amount that stands.

Proof sketch. (a) `accepted` is reachable only through `Acc` (Theorem 1), and `rejected` and `disputed` only through an admitted review carrying that decision; `Silent(L, c)` excludes all three. The remaining terminal transitions are the closing of the review window, the timeout and the withdrawal, and the first two set the form from `T` and read no Sigma result, value or quote. (b) Each guard compares the time of the head the closing references with a close computed from `T` and from the heads that date the event opening the window, reads no time from the closing event, and admits the fabric or either party as its emitter (§6.7); on a Case already terminal, which under `Silent(L, c)` is `withdrawn`, both are ignored (Theorem 3). (c) The fold admits a quote only from the quoter `T` names, only over the Case's Capsule, only under the digest `T` pins, only after the Sigma result it prices, and only when the earliest readable head that includes it leaves a full window before the Case's deadline, and keeps the first it admits; since the quote follows the Sigma result, the window on such a Case opens at the head that first includes the quote and closes by the deadline, so no admissible closing or timeout precedes it. ■

Part (a) is the formal content of the title. Every state a Case reaches through silence is `expired`, under a form the silent party signed before the Case opened. Part (b) says every window the terms state closes on a head, and anyone who can present the head, and whose event the fabric includes, can end it. Part (c) bounds what an uncontested credit can cost: the contract's price. What each form costs is read from `T` outside the fold, and money moves on it only with the payer's authorization (§7.7); the fold records the form and no amount.

6.11 Replay sufficiency.

> **Definition 1 (Replay Sufficiency).** Sigma is a pure function `σ(Claim, Capsule, Profile) → SigmaOutput × Readout`. A ReplayBundle `B` for Case `c` is *replay-sufficient* if it contains the transitive closure of `σ`'s inputs at evaluation time — the Claim, the admitted evidence set and the Profile artifacts themselves, not references to them — so that `σ(B)` reproduces the original pair for any evaluator holding `B` and nothing else.

Replay sufficiency is a construction property rather than an empirical one: the bundle is assembled by closing over σ's inputs. Closing over the Profile distinguishes an offline evaluator from one that must be trusted to hold the same artifact store. A settlement-log bundle carries the Case's settlement log, its heads, the registry and the standard, and an offline verifier folds it to the same state, deciding event and head. §9.1 tests our assembly of the bundle against the definition.

6.12 The model in the implementation. Every object above is implemented. The table below gives, for each, the number of tests in the implementation's suite that hold it, as pytest collects them, and the invariant harness counts its rows.

Theorem 4 and §6.8 rest partly outside the table. The fold holds what a fold can: it refuses to fold with heads that do not commit to the events supplied, or whose time decreases as the log grows, and under a Profile requiring receipts it reads no uncorroborated time. Whether participants gossip is a behaviour no fold can test, and §10.5 states the assumption the rest of the guarantee rests on.

Formal object	Tests
Fold F and the authority chain (§6.1, §6.2, §6.4)	22
Event admissibility, adversarial logs, silence (§6.2, §9.2)	61
ReviewAuthority(c), Acc (§6.3)	44
Grant constitution K, AdmitReview, arming and D(P) (§6.1, §6.3, §7.2)	45
Signed heads, proofs, receipts, the fabric's clock (§6.6, §6.8, §6.9)	33
Key versions key(p, t): rotation, retirement, recovery (§6.1)	15
The fabric's terminal power (§6.7)	19
Settlement terms: the review window, the expiry form, credit Claims, the quote (§5.7, §6.10)	61
Verification function σ, computed from evidence (§6.11, §7.5)	27
Corpus, cross-Profile pairs, replay, causal order, silence (§9, §9.1, §9.3)	8
Profile resolution and pinning (§7.2)	16
Offline verifier over settlement-log bundles (§6.11)	6
Remedy computation (§7.5)	19

The invariant harness exits non-zero when any row misbehaves, so a deployment can gate a release on it.

Two entries hold properties that prose alone would leave unchecked. The order invariance of §6.4 is held by folding every Case log of the corpus in several causally consistent orders (§9.1), and by a chain test that folds sampled linearizations of one log, without heads, to the same terminal state. The self-containment of Definition 1 is held by a test that empties the registry root, supplies the Profile artifacts with the bundle, and asserts the same Sigma result, reason code and replay digest as the same input resolved from the deployment's store.

6.13 Where verification-driven settlement sits. Instantiate the model with `ReviewAuthority(c) = {v}`, where `v` is the network's verifier, and let the verifier's output be admitted as a review event. Theorem 1 continues to hold, and it now guarantees that acceptance came from the verifier. The design of [3] is this instantiation, and its own treatment of human judgment shows why: a human arbiter enters as a verifier whose output is aggregated with the machine verifiers rather than as an authority whose signature disposes of the Case [3, §6.3]. The parameter is set in advance there as well: the Obligation Object the parties sign at binding carries the verifier policy, the settlement policy and the finality policy, so the parties chose the instantiation before execution [3, §4.2, §6.1]. The designs differ in where the

choice is written and in how it can be read and withdrawn: here it sits in the Profile the counterparty reads before agreeing to be measured, and any delegation under it is a grant on the Case's own log, bounded by `K` and revocable on the chain. The design of [19] is the same instantiation with the verifier reduced to a predicate over escrowed funds. The design of [20] is not an instantiation of it: its output is a proof that a stakeholder, an arbitrator or an insurer checks, and the decision stays with whoever checks it, outside the proof. In the terms of §5.4 it establishes evidence finality and measurement finality and leaves counterparty finality to an authority the proof does not name, which is the separation this paper argues for, reached with trusted hardware where §5.1 reaches it with contemporaneous commitment.

Our implementation refuses it. A review signed under a verifier or fabric role carries no standing and is inert at the fold, and `K.holders` names a party alone unless the Profile adds a role, so a deployment changes the parameter in the Profile, where the counterparty reads it. A deployment that wanted verification-driven settlement would need to grant the verifier review authority explicitly, in the Profile, where the counterparty can see it and decline.

The design of [3] and ours both compile obligations, bind evidence, and verify. They differ in the value of one parameter: whether the review authority is a party to the contract or a component of the network. Every argument in §5 is an argument about how that parameter should be set.

---

7. System Design

7.1 Overview. PSYCHE is organized as a layered pipeline over a single Case spine. Every participant sees a role-scoped projection of one Case, and the system maintains no per-role business facts.

```
Contract / Profile  →  Identity / Registry  →  Arming (co-signed)
     →  Evidence Ingress  →  Evidence Gate  →  Claim Binding
     →  Capsule / ReplayBundle  →  Sigma  →  VerificationReport
     →  Review | Expiry under the co-signed form
     →  Settlement Fold / Final State  →  Commercial Consumption
```

The canonical state flow:

```
Profile Armed, settlement terms co-signed
  → Claim Submitted
  → Evidence Admitted | Evidence Insufficient
    | Evidence Conflict | Profile Mismatch
  → Capsule / ReplayBundle Generated
  → Sigma result ∈ {claim-supported, claim-not-supported,
                    evidence-fault, requires-review}
  → VerificationReport Issued
  ================ authority boundary ================
  → claim_review ∈ {accept, reject, dispute, abstain}
    | review window closed: expired under the co-signed form
  → Signed Settlement Events Folded
  → final_state ∈ {accepted, rejected, disputed, expired, withdrawn}
  → Consumption: payment | credit | insurance | audit | archive
```

The marked edge is the only one in the graph that crosses from the evidentiary to the authoritative. One route across it reaches `accepted`, and it carries a review signed by the party the Case binds; the other reaches `expired`, at a cost that party signed at arming. Every decision below follows from keeping the edge singular.

7.2 Contract Profile. The Contract Profile is the compiled, machine-evaluable form of the agreement. It carries the obligation set; the evidence requirements per obligation, including required source classes and independence constraints; the performance metric with its measurement window and exclusion list; acceptance rules and review deadlines; authority limits per participant; payment terms; and the remedy schedule.

The Profile is pinned by content, before delivery. A Profile is a set of artifacts — the recipe, the parameter pack, the expected model, the spot-check policy — and arming one is an event that carries a single commitment to all four, under the Profile's name and version:

```
D(P) = H( canon( profile, version, H(recipe), H(pack), H(model), H(policy) ) )
```

Arming precedes claim submission and happens once for a Case. The Claim references the arming event, the Capsule carries `D(P)`, and σ refuses to evaluate — `capsule-invalid`, no result — unless the artifacts it resolves reproduce `D(P)`. An evaluation that names no standard is refused for the same reason: pinning that an input may decline to do is not pinning, since a result produced against an unpinned standard would be indistinguishable from one produced against a pinned one. A deployment that amends an artifact under an unchanged version identifier therefore does not silently change the standard for earlier Claims; it produces a mismatch that stops the evaluation.

Contemporaneous commitment enters the design here, on the same terms as it does for evidence. Neither party can select an interpretation of the metric after seeing how the window turned out, because the interpretation is committed by digest in an arming the heads date before the window. The fold recomputes the commitment from the recipe it is given and refuses a standard that does not reproduce the digest the arming carries. A dispute about the standard becomes a dispute about which arming event was folded, which the log answers under Theorem 5 and Corollary 5.1.

The arming is co-signed. One party signs the arming event and the other co-signs the same content. Besides `D(P)`, the arming carries the Case's review deadline and value at risk and the settlement terms of §5.7: the review window, the expiry form with its parameter, and the credit terms with the quoter and the digest of the commercial terms a credit is priced under. The fold refuses an original arming that lacks the co-signature, and one whose settlement terms carry a field it does not know; a credit Claim's arming carries the original arming in place of a co-signature (§5.7). A Claim that contradicts the armed deadline or value is ignored, so no later event moves a number a guard reads.

The Profile is privacy-partitioned. Commercial terms are frequently confidential, including from parties who must nonetheless participate in evaluation. The Profile separates evaluable structure from commercial terms, and digest binding establishes that the structure evaluated is the structure armed, without disclosing what was withheld.

	Operator	Customer	Evidence source	Verifier	Fabric
Obligation set	full	full	own class	full	digest
Metric, window, thresholds	full	full	own class	full	digest
Required evidence classes	full	full	own class	full	digest

	Operator	Customer	Evidence source	Verifier	Fabric
Price, credit schedule, caps	full	full	none	none	digest
Settlement terms	full	full	none	none	full
Authority limits	full	full	none	scope only	full
Evidence payloads	own	own	own	admitted set	digest

full is the whole object; *digest* is a commitment to it and nothing more; *own class* and *admitted set* are the partitions named in §7.3. Grants and the settlement terms are events on the Case log, so the fabric, and anyone holding the log, reads them; G6 depends on that. The price list the terms pin reaches the fabric as a digest.

The second row is the exposed one: verifiers learn thresholds, since they evaluate against them. In compute markets a negotiated bandwidth floor is competitively sensitive, and a verifier serving several providers accumulates a view of the market's terms. The present design mitigates by scoping verifiers per Case and withholding price, and it does not solve the problem. The setting is a commit-and-prove one — commit to the Profile, reveal only the predicate, prove evaluation ran against the commitment — and §11 treats it as open.

In the hosted configuration of §8 one deployment, operated by PSYCHE, runs the fabric, the verifier, the quoter and the evidence store. It files each tenant's evidence payloads under that tenant, encrypted under a data key its own key service issues, and can decrypt them; its verifier reads what the verifier's column shows, and its quoter reads the price list in full.

A further role reads without deciding. A **beneficiary** — a lender whose covenant tests on the obligation, an insurer, an auditor — is registered as a participant and reads the Cases a contracting party opens to it. The implementation seats three such readers (Financing Party, Rating Agency, and Exchange and Clearing), each under a disclosure grant: a record a contracting party's key signs, naming the holder, the contracts it covers and the period it runs, held apart from the Case log and revocable by the same key. A grant from either party opens what both parties filed on those Cases, so the contract's confidentiality terms decide whether one party's grant is enough. For each Case a grant covers, the reader receives the signed settlement log with its published heads, the Profile and the registry entries of its signers; the Capsule's references, digests and proofs; the Claim's value and the quotes and credits on the record; the price list as a digest; and the periodic attestations the parties file. It folds the Case's state from the log itself (§6.11) and, without the evidence payloads, reads the Sigma result as recorded. A beneficiary reads: `K.holders` names a party alone unless the Profile adds the role. Adding the role costs the model nothing, since it is read-side only and writes no event that δ folds. The party whose money moves on a delivery record need not be a party to it, and a lender or insurer cannot underwrite a record it cannot read.

7.3 Evidence Gate. The Evidence Gate determines admissibility, and here we diverge most sharply from monitoring-derived designs. It runs the following checks.

Check	Question	Adversary addressed
provenance	who produced this evidence	—
signature	is it signed, under which key, at what time	A4
time alignment	does the evidence window match the claim window	A2
completeness	is any hard-required evidence absent	A1
tamper evidence	is it hashable, chainable, verifiable	A4

Check	Question	Adversary addressed
source class	first-party, second-party, third-party, hardware, customer-side	—
independence	does the source set meet independence and strength floors	A9
conflict	do operator-side and customer-side evidence disagree	—
profile match	does the evidence satisfy the armed Profile's requirements	A2
coverage	does the evidence window cover the claim window	A1, A2
boundary materiality	are material gaps covered by a material BoundaryRecord	A1

Sigma evaluates the admitted evidence in six gates, after a resolve stage and a seal check, and every gate computes from what was observed: measured series, BoundaryRecords, envelopes and spot-check transcripts under the armed Profile. The first non-passing gate decides the result.

Gate	Decides	Result when it does not pass
resolve	the Capsule is well formed, and its standard, parameter pack and Sphinx log anchor reproduce	`capsule-invalid`, no evaluation
seal	the replay digest the Capsule declares reproduces from its input	`capsule-invalid`, no evaluation
gate	hard-required channels present, sampled enough, spot-checked as required	`evidence-fault`; `requires-review` below minimum sample count
coverage	the evidence window covers the claim window, gaps declared	`claim-not-supported`
xer	the evidence portfolio is complete and the parties' filings agree	`evidence-fault` when short; `requires-review` when the sides' filings diverge
cr	spot-check transcripts answer the spot check issued, in time	`claim-not-supported`; `requires-review` when answered late at grade 2, or missed at lower value at risk
rcs	the delivery is consistent with the claim under the pinned thresholds	`claim-not-supported` below the floor; `requires-review` at or above θ_soft or θ_U
boundary	each declared gap lies within the pack's tolerance	`claim-not-supported`

Absence is a result. An unobserved quantity is counted as unobserved. A hard-required channel with no data is an evidence fault. A gap inside the claim window that no BoundaryRecord declares leaves the Claim unsupported for the window it asserts, and a declared gap longer than the pack's tolerance does the same. The **BoundaryRecord** exists for this purpose alone: a signed, timed, attributable statement that no data exists for a named interval, and why.

Gaps get their own object because they carry most of the disagreement. In a market where degradation is continuous and instrumentation is partial, the intervals both parties can see rarely produce controversy, because the numbers agree. Controversy concentrates where one party has data and the other does not, or where neither does. A system without an honest object for a gap must place the gap somewhere, and the two available choices are to treat unobserved time as delivered or to treat it as failed. Each choice silently transfers

value, in a direction fixed by an implementation detail, in a quantity nobody negotiated. The BoundaryRecord makes the gap a named object that the parties argue about directly.

The Gate also handles divergence without adjudicating it. When the customer's filed series and the operator's give different levels for a hard-required channel, the Gate carries the divergence to decision: the Case returns `requires-review` with `XER.DIVERGENCE` as its reason, stating both figures, and the party with standing decides on a record that states both figures [29, H4, DR-06]. Choosing a figure — the lower, the mean, a capped one — would exceed the system's authority in the way this paper argues against. A counterparty can therefore bring a Case to `requires-review` by filing a diverging series. The series is signed and committed as it was measured, so a fabricated divergence is attributable to the party that filed it (§5.1), and the Case then waits for that party's own signature or expires under the form it signed. §10.9 states what remains.

7.4 Claim Binding and the Proof-of-Reality Capsule. A Delivery Claim has the canonical form:

> Operator A claims delivery of X units to Customer B during window T under Profile P and registry snapshot S, supported by evidence receipts E, subject to settlement policy R, counterparty review, and the final state the fold derives.

Under Invariant III the Claim carries no verdict and no status. A Case carries one Claim, and the Claim must name the claimant, counterparty and standard its arming names.

The **Proof-of-Reality Capsule** is the producer-signed package binding exactly one Claim to its admitted evidence. It carries content and references rather than bulk payloads: an XER envelope per external evidence receipt, the support artifact that attributes the receipt to a party and a trust root; the spot-check transcripts where Sphinx issued spot checks [17, 18], with the head of Sphinx's issuance log (`oracle_log_head_digest`) under which each spot check's issuance is proven by a Merkle path; BoundaryRecords for coverage gaps; seven reality-consistency digests, over the measurements, the envelopes, the transcripts, the BoundaryRecords, the log head, the parameter pack and the registry snapshot; a provenance digest over its receipts, their sources and its sealer; the verification material the evaluation is replayed from, with its replay digest; and a `parameter_pack_digest` acting as an equality tripwire against the registry snapshot.

The associated **ReplayBundle** satisfies Definition 1. An offline verifier — an audit firm, an insurer, the counterparty's own systems — recomputes without access to either party's live infrastructure, which removes the principal obstacle to third-party review in a setting where both parties treat their telemetry as sensitive.

The same property has a second consequence we did not design for. Because the bundle carries its Profile artifacts, its signatures and its inclusion proofs, it remains checkable when the systems that produced it are gone. The practice described in §2 holds no record with that property: reconciliation lives in the parties' consoles, and a party's failure takes its side of the arithmetic with it. In a market where providers may not outlive a five-year facility, a record has to survive its author.

Under Invariant II the Capsule carries no counterparty judgment.

7.5 Sigma. Sigma evaluates a Capsule against the armed Profile and emits one governed result.

Sigma result	Meaning
`claim-supported`	admitted evidence supports the Claim under the pinned standard
`claim-not-supported`	admitted evidence contradicts the Claim under the pinned standard
`evidence-fault`	hard-required evidence missing or unsatisfied; not evaluable
`requires-review`	evaluable only through a judgment the system lacks authority to make

A two-valued verifier has no way to say that it cannot tell. Presented with an evidence set that neither supports nor contradicts the Claim, it must return one of the two answers it holds, and whichever it returns assigns the loss to one of the parties. That choice is a commercial term: it decides who pays when the record is silent. Encoding it in a verifier's error handling buries a material contract term where neither party's counsel will read it, and the term then applies uniformly across every contract the system processes, regardless of what any of them say. §9.3 measures the size of this effect on our corpus.

Under PSYCHE the Cases a two-valued verifier would allocate by its error handling are decided by a review or by nobody: silence ends them in `expired`, under the form fixed before the result existed, as it ends every other Case (§5.7). §9.3 settles the corpus under settlement terms and shows no result, determinate or inconclusive, reaching a decision by silence.

Splitting the inconclusive results further separates two situations that look alike and route differently. Under `evidence-fault` the evidence is inadequate, and the remedy is collection or re-collection. Under `requires-review` the evidence is adequate and the question exceeds the system's authority, and the remedy is a review, the counterparty's or its delegate's. Collapsing these sends genuine authority questions into an evidence-collection loop that cannot resolve them.

Who bears an evidence fault. A1 gives a party the option of withholding evidence that would damage it, and the Gate's response is `evidence-fault`. Whether withholding is profitable therefore depends on what a fault costs, and that is a commercial question with no natural answer. If a fault suspends payment, the buyer gains by suppressing its own scheduler records. If a fault admits the Claim, the operator gains by suppressing its telemetry. The allocation belongs in the Profile as an explicit term, per obligation, negotiated like any other. Making the allocation visible does not remove the incentive, and it moves the incentive to a place where the parties priced it. A hard-required evidence class held by a party who benefits from its absence is a Profile defect detectable at arming time. In the present implementation the allocation term is a field of the contract's SQCD-1 addendum (J1), and the check runs when the addendum is authored; the Profile does not yet carry the term, and arming does not repeat the check (§10.1).

The **VerificationReport** renders the Sigma result into a multi-party-readable artifact carrying the `bundle_digest` it observed, so any party can confirm the report describes the bundle in their hands.

Aggregate intelligence derived across governed Cases is governed by Invariant VI and takes no part in any of the above.

7.6 Review, disputes and credit Claims. The counterparty, or an agent acting under an explicit and revocable delegation from the counterparty, signs a `claim_review` carrying a decision, the Claim and Capsule it binds, the grant under which it was made, and the authority event it follows.

The decision is signed, so it is attributable and non-repudiable. Binding the grant into the event gives G6 its force: a reader of the log can check the holder, the validity window, the ceiling and revocation without consulting the granting party, so overreach is detectable, and §6.2 makes it inert. `abstain` is available as a recorded value that leaves the Case where it stands: a reviewer can record participation without expressing a decision. Multi-reviewer commercial processes produce this situation constantly, and a three-valued model forces it into a distortion.

A reject and a dispute are terminal when signed. A `disputed` Case leaves the record as it stands, with the Capsule, the Sigma result and the signed dispute on it, and is resolved outside the fold, on a log either side can present. Nothing resolved there re-enters the Case: a shortfall established afterwards is claimed on a Case of its own.

A credit is claimed on a Case of its own. The counterparty arms `c/credit` by presenting the original Case's signed arming and makes the credit Claim before the credit terms' last date, as the earliest head that includes it shows. The verifier evaluates the credit Claim's Capsule under the original standard. The original claimant reviews; its silence leaves the credit Case to expire under the credit expiry form, and under EX-03 what stands is the quote the named quoter attested under the pinned price list.

7.7 Settlement fold and consumption. The terminal state is derived by the fold of §6.1. `expired` follows a window closing or a timeout, each admissible under §6.7 from the fabric or either party referencing a published head at or after the window's close or the Case's deadline, and the result names the form the Case expired under; `withdrawn` follows a terminal event only the claimant emits.

The deployment's services run a settlement clock that publishes heads and announces each window's close once a head shows it; either party can do the same from the fabric's heads. The services and the Terminal read every Case state from the fold, and none of them writes one.

Downstream systems subscribe to the terminal state and are fold-inert. Money follows the terminal state and the form. On a delivery Case the record releases the invoice on `accepted`, and on `expired` under EX-02, where the invoice stands; under EX-01, EX-03 and EX-04 it releases none. EX-01 leaves nothing owed and EX-04 the stated amount. The remedy EX-03 leaves standing takes an amount only on a credit Case, where the quoter prices it: a credit Case `expired` under EX-03 leaves the remedy standing at the quote, and a delivery Case with no credit Case, or a credit Case with no quote recorded, fixes no amount. `rejected` and `withdrawn` release nothing, and a `disputed` Case leaves the amount to a resolution outside the record (§7.6). Each moves only with the payer's own authorization. PSYCHE holds no funds and moves no money. It produces the record on which movement is authorized elsewhere, for the reason given in §5.4.

7.8 Transparency and gossip. The fabric publishes signed tree heads on the schedule the Profile requires, and serves inclusion proofs for any event and consistency proofs between any two heads. Participants store the heads they receive and sign receipts for them (§6.8).

The Profile specifies a gossip obligation: minimally, each participant presents its most recent head to the other party when submitting or reviewing. That obligation activates Theorem 4. An auditor may subscribe to heads, and receipt them, without holding any other role.

Head publication frequency is a Profile parameter, and it trades operational cost against the tightness of the temporal bound in Theorem 5 and the resolution of every window in §5.7. The §5.1 argument needs at least a head at the close of each measurement window. More frequent publication narrows the interval within which a backdated record could hide.

---

8. Implementation

The system implements the full pipeline of §7 in Python, with a browser client in HTML, CSS, JavaScript, and custom WebGL. The backend is event-driven around the canonical Case state machine. Every record is canonicalized under the digest suite `psyche.digestsuite/1` — canonical JSON with sorted keys and compact separators, ASCII-escaped — and bound by its SHA-256 digest into Case events, which each participant signs with the Ed25519 key the participant registry binds to its role and class. Read models are role-scoped projections rebuilt by replay, and no projection is authoritative.

The invariants of §5.3 are enforced structurally. The terminal state exposes no setter and is reachable only as the output of the fold, so a violation requires a change to the event model rather than a mistake in application code. Sigma is implemented as six gates over an evaluation context, after a resolve stage and a seal check, each gate computing from the admitted observations; the four values of §7.5 are the codomain of the six gates, and the resolve stage and the seal check refuse before they run. The deployment's services and its Terminal read every Case state from the fold, and a settlement clock in the services process publishes heads and announces each window's close (§7.7).

The properties of §6 are held by the suites §6.12 counts, and the harnesses of §9 emit every table in that section. The invariant harness exits non-zero on any violation, so a release can be gated on it. The wider suite is larger; only the suites §6.12 counts are cited.

Availability. The implementation is closed. The paper gives a reader the specification: the invariants of §5.3, the model of §6 and the design of §7, stated in enough detail to reimplement from, with §11 recording where they are incomplete. The figures in §9 are emitted by harnesses, so hand-entry is absent as a source of error in them, and a reader is left relying on our report of what those harnesses returned. §10.4 states the larger limitation: the corpus behind them is synthetic.

The production release candidate specifies an ASGI API surface under OpenAPI 3.1, a PostgreSQL event store with derived read models, durable workers driven by a transactional outbox, S3-compatible object storage for evidence payloads, OIDC for human participants and OAuth client credentials for machine participants, and OCI containers on infrastructure defined as code with managed KMS and OpenTelemetry. The build evaluated in §9 differs from this specification in the respects stated in §10.1.

---

9. Evaluation

Every table in this section is emitted by a harness. §8 states what a reader can check independently.

Four results carry the section, in descending order of what they establish. Identical admitted evidence is judged differently under two pinned standards, which shows that a delivery record carries no result of its own and takes one from the standard (§9). On a worked contract, the amount a two-valued verifier would allocate through its error handling on a Case it could not decide is half the contracted period (§9.4). Under settlement terms both parties signed, silence ended every Case in `expired` under the form the arming selected, whatever the verifier had returned, and reached acceptance in none (§9.3). And the enumerated invariants caught a defect in our own acceptance check (§9.2), an argument for enumerating invariants in place of intending them.

We evaluate five questions. Is the derivation reproducible from the replay bundle alone (§9.1)? Do the derivation invariants hold under adversarial logs, and does silence end a Case anywhere other than `expired` under the form its arming selected (§9.2)? How much does the four-valued output change outcomes against a two-valued baseline, and in whose favor (§9.3)? What does silence leave across the corpus under settlement terms (§9.3)? Does the pipeline reduce a realistic dispute to a tractable object (§9.4)? §9.5 characterizes where the cost of such a system sits.

Corpus. Every Case is evidence a producer could file: one day of fifteen-minute samples for every channel both Profiles require, the envelopes that attribute each series to a party and a trust root, and spot-check transcripts whose issuance is proven by a Merkle path under the head of Sphinx's log the Capsule carries. Sigma computes every gate from that evidence. A condition changes what was observed — withholds a series, thins it, degrades its level, deletes samples, files the customer's diverging series, answers a spot check late — and the gates compute from what was observed. Twenty-two conditions run: a clean delivery, sixteen aimed at one of the six gates, two placed in the band between the two Profiles' thresholds, and three refusals at the resolve stage. Several instantiate adversary capabilities of §3.2 directly: the missing hard-required channel is A1, and the under-covering evidence window is A2.

Each condition is generated 40 times under each of the two Profiles, so a condition row carries 80 Cases and the corpus carries 1,760. Each Case's clean evidence is drawn from its own seed, so no two Cases share an input, and the same Case under the two Profiles carries byte-identical evidence. Cells of equal size make the corpus a coverage instrument rather than a sample. Aggregate rates over it describe the generator. Per-condition rates are the meaningful figures, and §10.4 states why neither can be read as a rate over real deliveries.

Condition	Aims at	Cases	Distinct	Invalid	Supported	Not supported	Fault	Review	Off target
Clean delivery	none · `SIGMA.SUPPORTED`	80	80	0	80	0	0	0	0
Hard-required channel missing	gate · `GATE.EVIDENCE_FAULT`	80	80	0	0	0	80	0	0
Required channel unavailable	gate · `GATE.EVIDENCE_FAULT`	80	80	0	0	0	80	0	0

Condition	Aims at	Cases	Distinct	Invalid	Supported	Not supported	Fault	Review	Off target
Required spot check missing	gate · `GATE.REQUIRED_CHALLENGE_MISSING`	80	80	0	0	0	80	0	0
Channel below minimum sample count	gate · `GATE.WARMING`	80	80	0	0	0	0	80	0
Evidence window under-covers claim window	coverage · `COVERAGE.UNDER_COVER`	80	80	0	0	80	0	0	0
In-window gap with no BoundaryRecord	coverage · `COVERAGE.GAP_NO_BR`	80	80	0	0	80	0	0	0
Required evidence item absent	xer · `XER13_PORTFOLIO`	80	80	0	0	0	80	0	0
Sides' filings diverge	xer · `XER.DIVERGENCE`	80	80	0	0	0	0	80	0
Answer addresses another spot check	cr · `CR.CORRUPTED`	80	80	0	0	80	0	0	0
Spot check answered before issuance	cr · `CR.IMPOSSIBLE`	80	80	0	0	80	0	0	0
Spot check answered late at grade 2	cr · `CR.LATE_G2`	80	80	0	0	0	0	80	0
Attested identity differs from the claimed	rcs · `RCS.FLOOR`	80	80	0	0	80	0	0	0
Shortfall burden above theta_soft	rcs · `RCS.SOFT`	80	80	0	0	0	0	80	0
Uncertainty above theta_U	rcs · `RCS.UNCERTAIN`	80	80	0	0	0	0	80	0
Required channel uncalibrated	rcs · `RCS.UNCERTAIN`	80	80	0	0	0	0	80	0
Burden inside the inter-Profile band	none · `SIGMA.SUPPORTED` / rcs · `RCS.SOFT`	80	80	0	40	0	0	40	0
Uncertainty inside the inter-Profile band	none · `SIGMA.SUPPORTED` / rcs · `RCS.UNCERTAIN`	80	80	0	40	0	0	40	0
Declared gap longer than pack tolerance	boundary · `COVERAGE.GAP_NO_BR`	80	80	0	0	80	0	0	0
Parameter pack equality tripwire	resolve · `RESOLVE.PACK_DIGEST_MISMATCH`	80	80	80	0	0	0	0	0
Transcript issuance not proven in the oracle log	resolve · `RESOLVE.ANCHOR_MISMATCH`	80	80	80	0	0	0	0	0
Asserted registry snapshot digest is false	resolve · `RESOLVE.SNAPSHOT_MISMATCH`	80	80	80	0	0	0	0	0
Total		**1,760**	**1,760**	**240**	**160**	**480**	**320**	**560**	**0**

Distinct counts distinct inputs. *Invalid* is `capsule-invalid`, *Fault* `evidence-fault`, and *Review* `requires-review`. Each condition names the gate and reason code it aims at. The off-target column counts Cases that answered anywhere else, including Cases every gate passed. It is zero throughout, so each condition reaches the gate it aims at. Conditions that share a gate and reason code are told apart by the evidence pointer and detail Sigma reports.

Two Profiles. The corpus runs in full under each of two armed Profiles, one for software performance warranty and one for compute delivery. They differ in required channels, evidence classes, spot-check types, and thresholds. Two Profiles separate what the evaluation pipeline decides from what a deployment's own standard decides.

Most conditions agree across Profiles, which locates their result in the gate. Two conditions are constructed to disagree: they place the reality-consistency burden and the uncertainty term inside the band lying between the two Profiles' thresholds, so the same evidence sits above one Profile's threshold and below the other's.

Condition	`saas-performance-warranty/1`	`compute-delivery-warranty/1`	Agrees
Clean delivery	claim-supported	claim-supported	yes
Hard-required channel missing	evidence-fault	evidence-fault	yes
Required channel unavailable	evidence-fault	evidence-fault	yes
Required spot check missing	evidence-fault	evidence-fault	yes
Channel below minimum sample count	requires-review	requires-review	yes
Evidence window under-covers claim window	claim-not-supported	claim-not-supported	yes
In-window gap with no BoundaryRecord	claim-not-supported	claim-not-supported	yes
Required evidence item absent	evidence-fault	evidence-fault	yes
Sides' filings diverge	requires-review	requires-review	yes
Answer addresses another spot check	claim-not-supported	claim-not-supported	yes
Spot check answered before issuance	claim-not-supported	claim-not-supported	yes
Spot check answered late at grade 2	requires-review	requires-review	yes
Attested identity differs from the claimed	claim-not-supported	claim-not-supported	yes
Shortfall burden above theta_soft	requires-review	requires-review	yes
Uncertainty above theta_U	requires-review	requires-review	yes
Required channel uncalibrated	requires-review	requires-review	yes
Burden inside the inter-Profile band	claim-supported	requires-review	**no**
Uncertainty inside the inter-Profile band	claim-supported	requires-review	**no**
Declared gap longer than pack tolerance	claim-not-supported	claim-not-supported	yes
Parameter pack equality tripwire	capsule-invalid	capsule-invalid	yes
Transcript issuance not proven in the oracle log	capsule-invalid	capsule-invalid	yes
Asserted registry snapshot digest is false	capsule-invalid	capsule-invalid	yes

Every cross-Profile pair carries byte-identical evidence. Under that identical evidence, a Case in the band returns `claim-supported` under one Profile and `requires-review` under the other. The same admitted set means one thing under one pinned standard and another under a second, and the difference lies entirely in terms the parties negotiated. A verifier whose thresholds belong to the verifier rather than to the contract gives every customer the same answer and lets none of them argue with it. Here the answer moves when the contract moves, and a party who disagrees has a term to point at.

9.1 Deterministic replay. Method. Five checks, each over every Case in the corpus. Every Case is written to disk as a verification bundle carrying its Sigma input and the Profile artifacts as values; a new interpreter, started with the deployment's profile-record store pointed at a directory that does not exist, verifies each bundle, and its replay digest, Sigma result and verification-report digest must equal the originals. The Case's evidence is presented again with every collection shuffled, and the Sigma result, reason code and gate map stay the same. Each Case is settled on a signed, chained Case log — the Claim, the Capsule, the Sigma result, a review whose decision follows the result, and records that depend on nothing; on alternate supported Cases the review is a machine delegate's, under a grant within an armed constitution. The harness derives $<$ from the log itself, folds the log's own order

and a seeded sample of the other orders consistent with <, each under a head the fabric signs over it, and requires the same state, deciding event, grant, accepting signers, forks, expiry form and quote from every order. Finally each Case log is written to disk with its heads, registry and standard, and the fresh process must fold it to the same state, deciding event and head.

Check	Cases	Identical
Sigma recomputed from the serialized bundle in a fresh process with no profile-record store, against the original computed from the store: replay digest and assertion	1760	1760
Verification report digest over the same bundle, originating and fresh process	1760	1760
Evidence presented with every collection shuffled: assertion, reason code and gate map	1760	1760
Case log folded in 10,560 causally consistent orders, 6 per Case: the log's own order and a seeded sample of the 20 to 168 each Case log admits. State, deciding event, grant, accepting signers, forks, expiry form, quote	1760	1760
Case log folded from the serialized bundle in the fresh process: state, deciding event, head	1760	1760

Result. Every check is identical across the corpus. The implementation satisfies Definition 1: recomputation in a fresh process from the bundle alone reproduces the evaluation bit for bit, including the digest chain, and the Case log folds to the same record there. An audit firm or an insurer can therefore evaluate a Case without touching either party's infrastructure. Order invariance is the empirical counterpart of Theorem 2b, and it lets two parties consuming the same events over different network paths agree without coordinating. The corpus logs carry one head and no closings or timeouts, so no time-bound guard is exercised here; §9.2 and §9.3 exercise them. Revocations and forks are held by the authority-chain suite of §6.12.

9.2 Invariant enforcement. Method. §9.2 drives the settlement fold. Every event in every row is signed with its signer's key and checked by the fold against the participant registry, and where a row turns on time the fabric signs heads over the log and the fold reads time off them and off nothing else. The harness folds the lawful path, then the adversarial configurations of §3.2; for the constitution of §6.1, where the enumeration is of routes by which a delegate could come to hold more authority than the Profile conferred; and for settlement terms co-signed at arming, where the enumeration is of routes by which silence, a closing or a quote could come to produce a decision. Rows in the last group expect a named state, and where that state is `expired`, the form the Case expired under.

Five rows are positive controls: a lawful counterparty accept; a lawful delegated accept inside the ceiling; a delegated accept inside a grant's window under a Profile whose heads an auditor must corroborate; a machine accepting `requires-review` under a constitution widened to admit it; and a counterparty's accept inside the review window it co-signed. A table reporting that nothing derived `accepted` establishes nothing unless something can, and the harness refuses to report the negative rows as a result when a control fails.

Group	Subject	Configuration	Final state	Derived `accepted`	As specified
Positive control	Counterparty-signed accept, in authority	the lawful path, signed and chained	`accepted`	**yes**	yes
Adversarial configuration	Self-accept	the claimant reviews its own claim (A7)	`sigma_recorded`	no	yes
Adversarial configuration	Role spoofing in the payload	claimant signs, counterparty identity inside (A7)	`sigma_recorded`	no	yes
Adversarial configuration	Forged signature	the counterparty's review, signed with another key	`sigma_recorded`	no	yes

Group	Subject	Configuration	Final state	Derived `accepted`	As specified
Adversarial configuration	Unregistered key	a fresh identity declaring itself counterparty	`sigma_recorded`	no	yes
Adversarial configuration	Party the Claim does not name	a registered party declaring review standing	`sigma_recorded`	no	yes
Adversarial configuration	Unbound review	accept that binds neither claim nor capsule digest	`sigma_recorded`	no	yes
Adversarial configuration	Verifier signs the review	the network's verifier as review authority	`sigma_recorded`	no	yes
Adversarial configuration	Replayed accept after rejection	a stale accept re-submitted post-settlement (A8)	`rejected`	no	yes
Adversarial configuration	Payment after rejection	funds move, then the state is asked to follow	`rejected`	no	yes
Adversarial configuration	Second Sigma result from the claimant	the assertion a machine delegate reads, rewritten	`sigma_recorded`	no	yes
Adversarial configuration	Second Claim	a later Claim lowering the value under a grant cap	`sigma_recorded`	no	yes
Delegated authority (§6.1)	Lawful delegated accept	a machine holder, inside the ceiling	`accepted`	**yes**	yes
Delegated authority (§6.1)	Lawful accept inside a grant window	signed heads, corroborated by an auditor's receipts	`accepted`	**yes**	yes
Delegated authority (§6.1)	Widened constitution, machine accepts `requires-review`	the collapse, written into the Profile where it can be read	`accepted`	**yes**	yes
Delegated authority (§6.1)	Constitution in the arming event	a ceiling asserted beside `D(P)` instead of inside it	`no_claim`	no	yes
Delegated authority (§6.1)	Profile silent on delegation	no constitution in the recipe at all	`sigma_recorded`	no	yes
Delegated authority (§6.1)	Machine holder against `requires-review`	the §5.6 collapse, sought through an ordinary grant	`sigma_recorded`	no	yes
Delegated authority (§6.1)	Machine grant admitting nothing	an empty list of admitted outputs	`sigma_recorded`	no	yes
Delegated authority (§6.1)	Issuer calls its machine a human	`holder_is_machine: false` on the payload	`sigma_recorded`	no	yes
Delegated authority (§6.1)	Verifier as grant holder	§6.13's instantiation by ordinary grant	`sigma_recorded`	no	yes
Delegated authority (§6.1)	Claimant confers review authority	the claimant issuing its own reviewer	`sigma_recorded`	no	yes
Delegated authority (§6.1)	Windowed grant, review names no head	A3 on the window: the reviewer's own clock and nothing else	`sigma_recorded`	no	yes
Delegated authority (§6.1)	Head published after the window closes	a signed head, and the window shut before it	`sigma_recorded`	no	yes
Delegated authority (§6.1)	Stale head carried past the close	names a head inside the window; first included by a head after it	`sigma_recorded`	no	yes
Delegated authority (§6.1)	Backdated covering head, witnessed Profile	the fabric dates the review inside the window; the auditor received it late	`sigma_recorded`	no	yes
Delegated authority (§6.1)	Capped grant, Case fixes no value	a ceiling cleared by declaring nothing	`sigma_recorded`	no	yes
Delegated authority (§6.1)	Delegate names the pre-revocation link	a review forking the chain around a revocation	`sigma_recorded`	no	yes
Delegated authority (§6.1)	Review after revocation	the grant withdrawn earlier in the chain	`sigma_recorded`	no	yes
Silence under settlement terms (§6.10)	Accept inside the review window	the counterparty signs before the window the terms fix has closed	`accepted`	**yes**	yes

Group	Subject	Configuration	Final state	Derived `accepted`	As specified
Silence under settlement terms (§6.10)	Silence on a supported Claim	the review window closes on a published head; the Case expires under the form	`expired` under EX-02	no	yes
Silence under settlement terms (§6.10)	Silence on an unsupported Claim	the same window, closed on the same head	`expired` under EX-02	no	yes
Silence under settlement terms (§6.10)	Silence on `requires-review`	an inconclusive result, the window closed	`expired` under EX-02	no	yes
Silence under settlement terms (§6.10)	Silence on `evidence-fault`	an inconclusive result, the window closed	`expired` under EX-02	no	yes
Silence under settlement terms (§6.10)	EX-01 selected at arming	nothing owed on the claim either way	`expired` under EX-01	no	yes
Silence under settlement terms (§6.10)	EX-03 selected at arming	the remedy stands, uncontested	`expired` under EX-03	no	yes
Silence under settlement terms (§6.10)	EX-04 selected at arming	a stated cost, fixed in advance	`expired` under EX-04	no	yes
Silence under settlement terms (§6.10)	Window closed before it closes	a head published inside the review window	`sigma_recorded`	no	yes
Silence under settlement terms (§6.10)	Window closed with no head	the emitter's word that the window closed	`sigma_recorded`	no	yes
Silence under settlement terms (§6.10)	Verifier closes the window	the network's verifier as emitter	`sigma_recorded`	no	yes
Silence under settlement terms (§6.10)	Claimant closes the window	either party may present the head	`expired` under EX-02	no	yes
Silence under settlement terms (§6.10)	Counterparty closes the window	the silent party itself presents the head	`expired` under EX-02	no	yes
Silence under settlement terms (§6.10)	Terms signed by one party	the counterparty co-signs its own arming; nothing built on it stands	`no_claim`	no	yes
Silence under settlement terms (§6.10)	Timeout at the Case deadline	the deadline falls inside the review window; the fabric expires the Case	`expired` under EX-02	no	yes
Silence under settlement terms (§6.10)	Reject, then the window closes	a reject is terminal at once	`rejected`	no	yes
Silence under settlement terms (§6.10)	Dispute, then the window closes	a dispute is terminal at once	`disputed`	no	yes
Silence under settlement terms (§6.10)	Accept after the window closed	a decision given too late	`expired` under EX-02	no	yes
Silence under settlement terms (§6.10)	Terms carrying a default ceiling	an arming that would let silence decide; nothing built on it stands	`no_claim`	no	yes
Silence under settlement terms (§6.10)	Terms naming an arbiter	an arming that would seat a third decider; nothing built on it stands	`no_claim`	no	yes
Silence under settlement terms (§6.10)	Silent claimant on a credit Claim	a credit Claim priced by the named quoter under the pinned terms; the window runs from the head that first includes the quote	`expired` under EX-03, quote 80,000	no	yes

Group	Subject	Configuration	Final state	Derived `accepted`	As specified
Silence under settlement terms (§6.10)	Credit Claim asks more than the quote	the Claim asks 120,000; the quote attests 80,000; the quote is what stands	`expired` under EX-03, quote 80,000	no	yes
Silence under settlement terms (§6.10)	Window closed before the quote	a closing on a credit Case whose terms name a quoter, before the quote is recorded; no window has opened	`sigma_recorded`	no	yes
Silence under settlement terms (§6.10)	Quote after a refused closing	the quote arrives after a closing that found no window open; the window runs from the quote's head and closes with the quote standing	`expired` under EX-03, quote 80,000	no	yes
Silence under settlement terms (§6.10)	Quote before the Sigma result	a quote recorded before the Sigma result it prices is not a quote on this Case; the quote after the result is, and the window runs from its head	`expired` under EX-03, quote 80,000	no	yes
Silence under settlement terms (§6.10)	Quote in time, then the timeout	the quote is sequenced with a full window before the credit deadline; the timeout at the deadline leaves it standing	`expired` under EX-03, quote 80,000	no	yes
Silence under settlement terms (§6.10)	Quote sequenced at the deadline	the earliest head that includes the quote leaves no window before the credit deadline; nothing stands, whichever of the quote and the timeout the fabric folded first	`expired` under EX-03	no	yes
Silence under settlement terms (§6.10)	Quote under other terms	a quote priced under terms the parties did not pin is not a quote on this Case; no window opens, and the Case expires at its deadline with nothing standing	`expired` under EX-03	no	yes
Silence under settlement terms (§6.10)	Quote from the claimant	the credit claimant prices its own Claim; no window opens, and the Case expires at its deadline with nothing standing	`expired` under EX-03	no	yes
Silence under settlement terms (§6.10)	Original claimant rejects the credit Claim	the party the credit Claim binds decides it	`rejected`, quote 80,000	no	yes
Silence under settlement terms (§6.10)	Credit Claim brought by the original claimant	a credit Claim is the counterparty's; nothing built on it stands	`no_claim`	no	yes
Silence under settlement terms (§6.10)	Credit Claim after claims_until	the earliest head that includes it is too late	`no_claim`	no	yes

Result. All five controls settle. Self-accept, role spoofing between the event's signer and the payload's reviewer, a forged signature, an unregistered key, a party the Claim does not name, a review binding neither Claim nor Capsule, and a review signed by the verifier each fail to settle. A replayed accept after rejection is ignored under finality, as is a payment recorded after rejection. A second Sigma result signed by the claimant is refused, since only the verifier records one, and a second Claim lowering the value is ignored, since the Case has one.

The delegated rows close the routes around the ceiling. A constitution asserted in the arm-

ing event is refused, since a ceiling beside `D(P)` is not a ceiling the standard commits to. A Profile silent on delegation admits none. A machine holder accepts against `requires-review` where the Profile widens `machine_outputs`, and under a grant admitting a result; an issuer describing that machine as human changes nothing, because the class is read from the registry. The network's verifier holds review authority where the Profile names it, and review authority is conferred by the counterparty. A review under a windowed grant names a published head; the window closes on the head that closes it; a review naming a head from inside the window but first included by a head after it is refused; and under a witnessed Profile, a covering head the auditor received after the window closed is not read. A grant carrying a value ceiling refuses a Case that fixes no value. A delegate naming the link before a revocation forks the chain and is refused, as is a review after revocation.

The silence rows close the routes around the terms. Silence on a supported, an unsupported, a `requires-review` and an `evidence-fault` result ends in `expired` under the same form, and for each of the four forms the Case expires under the one its arming selected. A window closed on a head published inside it, closed with no head, or closed by the verifier ends nothing; closed by either party on a head at or after its close, it ends the Case. An arming the counterparty co-signed for itself arms nothing, and neither does one whose terms carry a ceiling up to which silence would accept, or a third party seated to decide. A timeout at the Case deadline expires the Case under the form. A reject and a dispute are terminal when signed, and a closing after either changes nothing; an accept after the window closed arrives on an expired Case and is ignored. A silent claimant leaves a credit Claim to expire under EX-03 with the quote standing, and a credit Claim that asks more than the quote expires with the same quote standing; a window closed before the quote is recorded ends nothing, and once the quote is on a head the window runs from that head and closes with the quote standing, whichever of the closing and the quote reached the fabric first; a quote recorded before the Sigma result it prices is not a quote on this Case, and the quote after the result is; a quote sequenced with a full window before the credit deadline stands through a timeout at the deadline, and one first included at the deadline is not a quote on this Case, so nothing stands whichever of it and the timeout the fabric folded first; a quote priced under terms the parties did not pin, or attested by the credit claimant, is not recorded, so no review window opens and the Case ends at its deadline with nothing standing; the original claimant's reject is terminal; and a credit Claim brought by the original claimant, or after the date the terms state, arms nothing.

The controls that settle are the ones §5.6 and §5.7 rest on. A delegated accept inside the ceiling works, so the refusals above refuse something specific. A machine accepting `requires-review` works once the constitution admits it, and the counterparty's own accept inside the window it signed works. Each way of letting something automatic stand in for a review is available, and each is reached only by saying so in advance, in a document the counterparty reads or signs and may decline. The one thing no document can say in advance is that silence accepts: the terms have no field for it, and an arming that carries one arms nothing.

What the enumeration caught. One row of this table did not behave as specified when it was first run. The acceptance check disqualified the claimant and admitted everyone else, so a review signed under the verifier role settled the Case. The finding is narrow: a missing positive condition in one access rule, in an implementation whose design argues against the arrangement the omission permitted. A system that grants verifier-held authority deliberately remains internally consistent, and §6.13 says so. The rule had been written as an

exclusion, *not the claimant*, where the property required is a standing, *holds review authority on this Case*. An exclusion silently admits every role defined afterwards. Stating it positively closes it, and the row is now a regression test. §5.3 claims that an enumerated, machine-checked invariant is stronger than an architectural intention, and this row is an instance in which the enumeration found a defect before anyone else did.

9.3 Four-valued against two-valued verification. §7.5 argues that a two-valued verifier encodes a commercial term in its error handling. This experiment measures the term, and then settles the corpus under terms the parties wrote themselves.

Method. We run the same corpus through the four-valued Sigma and through two ablations that collapse the inconclusive region: **B-supported**, mapping both `evidence-fault` and `requires-review` to `claim-supported`; and **B-not-supported**, mapping both to `claim-not-supported`. We report the number of Cases whose outcome differs from the four-valued run, and which party each collapse favors.

Baseline	Collapse	Cases diverging	Favors
B-supported	evidence-fault, requires-review → claim-supported	880	operator
B-not-supported	evidence-fault, requires-review → claim-not-supported	880	customer

The count is a property of the corpus mix and not a rate. The corpus weights every condition equally, so the figure says how many of *these* cells fall inconclusive and nothing about how often the situation arises in a deployment.

Reading. The divergence set is the same under both collapses: the Cases the four-valued engine declined to decide. Only the direction of transfer differs. A two-valued verifier therefore allocates that set through its error handling, and no negotiation chose the allocation. Three things generalize beyond the corpus: that the set exists, the direction of transfer under each collapse, and the per-condition breakdown below. The two collapses bracket the range: a Case in the set is worth the claimed quantity to the operator under one collapse and nothing under the other, with no term in any contract selecting between them.

Priced. Every inconclusive Case under the compute Profile, which carries commercial terms, is priced from the per-channel costs Sigma computed. The spread is the amount a two-valued verifier hands to one party or the other.

Inconclusive Cases priced	Spread min	Median	Max	Total
480	160,000	320,000	320,000	145,120,000

Under a contracted period priced at 320,000 minor units, the smallest spread is half the period and the median is the whole of it. Each Case's evidence covers one day, and the terms price its shortfall against the whole contracted period, so the spreads read the price list and not a day's delivery. §9.4 traces one Case at the smallest spread. The price list is ours, and §10.3 states what that leaves open.

Condition	Evaluable	evidence-fault	requires-review	Inconclusive share	Median spread
Clean delivery	80	0	0	0.0%	—
Hard-required channel missing	80	80	0	100.0%	320,000
Required channel unavailable	80	80	0	100.0%	320,000
Required spot check missing	80	80	0	100.0%	320,000
Channel below minimum sample count	80	0	80	100.0%	320,000
Evidence window under-covers claim window	80	0	0	0.0%	—
In-window gap with no BoundaryRecord	80	0	0	0.0%	—
Required evidence item absent	80	80	0	100.0%	320,000
Sides' filings diverge	80	0	80	100.0%	320,000
Answer addresses another spot check	80	0	0	0.0%	—
Spot check answered before issuance	80	0	0	0.0%	—
Spot check answered late at grade 2	80	0	80	100.0%	320,000
Attested identity differs from the claimed	80	0	0	0.0%	—
Shortfall burden above theta_soft	80	0	80	100.0%	160,000
Uncertainty above theta_U	80	0	80	100.0%	320,000
Required channel uncalibrated	80	0	80	100.0%	320,000
Burden inside the inter-Profile band	80	0	40	50.0%	240,000
Uncertainty inside the inter-Profile band	80	0	40	50.0%	320,000
Declared gap longer than pack tolerance	80	0	0	0.0%	—

Conditions differ sharply in how often they fall inconclusive. A two-valued verifier is biased in proportion to how often a given failure mode occurs in a given deployment. That frequency is a fact about the deployment's Profile and instrumentation rather than about the verifier. A deployment whose instrumentation fails mostly in ways that land at 0.0% loses nothing to a two-valued verifier; one whose failures land in the rows at 100.0% is transferring value on every such Case, in a direction its contract left unstated.

Silence under settlement terms. The same corpus, settled a second way. Each Case is armed with settlement terms the counterparty signs and the claimant co-signs: a review window, an expiry form, and credit terms naming a quoter and the pinned price list. Nobody reviews. The fabric closes the review window on a head published after it closes; a Case with no Sigma result opens no review window and expires at its deadline. Every Case runs once under EX-02, the invoice standing, and once under EX-03, the remedy standing, so the table shows the fold carrying the form the parties selected and the same terminal state under either. No credit Claim is brought, so the run measures the terminal state and the form and prices no remedy. The table counts Case logs, two per Case.

Sigma result	Case logs	Window open after the Sigma result	Final state	Expired under	`accepted` or `rejected`
`claim-supported`	320	review, closes at 110, into `expired`	`expired` 320	`EX-02` 160; `EX-03` 160	0
`claim-not-supported`	960	review, closes at 110, into `expired`	`expired` 960	`EX-02` 480; `EX-03` 480	0
`evidence-fault`	640	review, closes at 110, into `expired`	`expired` 640	`EX-02` 320; `EX-03` 320	0
`requires-review`	1,120	review, closes at 110, into `expired`	`expired` 1,120	`EX-02` 560; `EX-03` 560	0
none (`capsule-invalid`)	480	case, closes at 10000, into `expired`	`expired` 480	`EX-02` 240; `EX-03` 240	0

Every Case with a Sigma result expired when its review window closed, under the form its

arming selected: 1,760 logs under EX-02 and 1,760 under EX-03, 3,520 in all, every one of them `expired`. The 160 supported Cases expired alongside the 480 unsupported and the 880 inconclusive ones, and the 240 Cases with no Sigma result opened no review window and expired at their deadline. No log reached `accepted` or `rejected`. The set a two-valued verifier would allocate by its error handling is left, like every other Case, to a signature given at the time, and what the absence of that signature costs was signed in advance.

9.4 Worked example: one delivery, four outcomes. The trace below is generated by the harness. Every figure comes from the shipped evaluation functions applied to the armed Profile's own parameter pack, and the Sigma results come from the evaluation engine.

Setup. A Profile is armed for compute delivery over a one-day window sampled every fifteen minutes, under a price list for a contracted period of 1,000 H100-hours, with two hard-required channels — availability and memory bandwidth — carrying distinct weights, measurement tolerances, and caps, and requiring evidence from operator telemetry and customer-side scheduler records at independent trust roots. Four variants of one delivery differ only in what the admitted evidence shows.

Variant	Burden	RCS	RCS state	Sigma result	Deciding gate
Nominal delivery	42,000	958,000	consistent	`claim-supported`	none
Bandwidth below the contracted level	444,941	555,059	inconsistent-soft	`requires-review`	rcs
In-window gap, no BoundaryRecord	42,000	958,000	consistent	`claim-not-supported`	coverage
Operator telemetry withheld	—	—	—	`evidence-fault`	gate

Trace. Under the nominal variant both channels fall short of the contracted rate. The bandwidth shortfall lies inside the pinned tolerance and contributes nothing; the availability shortfall carries no tolerance and contributes its weighted cost. The aggregate sits below the Profile's soft threshold and the Case returns `claim-supported`.

Under the degraded variant both channels fall short by more. Availability's weighted cost alone now exceeds the soft threshold, and the bandwidth residual clears its tolerance and adds its own. The Case returns `requires-review`. Nothing about the evidence is defective; the standard has been applied, and the system has no authority to dispose of this delivery.

The third variant leaves the performance figures at their nominal values and adds an undeclared six-hour gap. Every gate runs, and the first in canonical order that does not pass decides. Coverage comes before consistency, so the Case returns `claim-not-supported` with its consistency score unchanged: the evidence that supported the Claim a moment ago no longer does, because part of the window is unaccounted for and no BoundaryRecord says so. The fourth variant withholds the operator's telemetry envelope, so neither hard-required channel has an admitted series, and returns `evidence-fault` at the first gate.

Pricing. The same four variants, priced against the Profile's commercial terms. The terms — credit schedule, unit price, cap, eligibility floor and the shortfall basis — live in a store of their own, apart from the evaluation registry, and every quote carries their digest, so the figures below were unavailable to the verifier that produced the Sigma results above. The two collapse columns price each Case under the two determinate results a two-valued verifier would have to choose between.

Variant	Quote status	Remedy type	Credit	Payable	If collapsed to supported	If collapsed to not-supported	Spread
Nominal delivery	not_eligible	service_credit	0	320,000	320,000	0	320,000
Bandwidth below the contracted level	withheld	withheld	—	—	160,000	0	160,000
In-window gap, no BoundaryRecord	eligible	claim_refused	0	0	320,000	0	320,000
Operator telemetry withheld	withheld	withheld	—	—	320,000	0	320,000

The withheld rows are the Cases the evaluation declined to decide, and they carry no amount, because a Case has an amount only once it has a determinate result. On those rows the spread column shows what a verifier without an inconclusive answer would allocate, in money, on the strength of its error handling. On the determinate rows the verifier has an answer, and the spread is not in play.

The Case halts here. Evidence finality holds on every variant. Measurement finality, in the two halves §5.4 names, holds in full on the nominal and the gap variants, which carry a determinate result and a price; on `requires-review` its first half holds, the shortfall is computed, and its second half waits, the credit a review would fix lying between the two collapse columns; on `evidence-fault` neither half holds, since the gate admitted no measurement. Nothing is settled on any of them. Under terms like those of §9.3, the customer's silence would leave every variant to expire under the one form it signed when the Case was armed; the Sigma result changes what the record shows and nothing about which form applies.

Interpretation. The instructive variant is the second. A verifier returning a binary answer would convert `requires-review` into support or into refusal, and the conversion moves half the contract value to one party on the strength of its error handling. Under the four-valued output the Case arrives at the reviewer with the reason named, the threshold it crossed identified, the amount at stake computed under both readings, and the arithmetic recomputable from the bundle by either side. The parties may still disagree. They now disagree about one number and can both see how it was reached.

9.5 Where the cost sits. Capsule assembly is linear in the number of evidence receipts and dominated by digest computation over reference sets rather than payloads. Sigma evaluation is linear in the number of Profile obligations times the number of admitted evidence intervals, with no cross-Case dependencies, so Case evaluation is embarrassingly parallel. Replay of a Case history and projection rebuild are linear in event count. Merkle inclusion and consistency proofs are logarithmic in log size. At the volumes implied by enterprise compute contracting — thousands of Cases per day rather than millions per second — none of this is computationally interesting.

The cost sits in evidence ingestion. Each new evidence source class requires an adapter normalizing a distinct telemetry format, clock discipline, and aggregation semantics into the admissibility model, and that cost scales with the number of source classes rather than with transaction volume.

A design premised on many heterogeneous evidence sources is therefore expensive before it is useful, in proportion to how many parties must be integrated before the first Case can run. §5.1 argues that the parties' own signed observations suffice, and §6.6 establishes that a deployment hosted by one of them remains accountable to the other. Two source classes

therefore constitute a working configuration, both of them already instrumented by parties with a reason to care about the outcome. A network of evidence providers raises the strength class of the admitted set and is not a precondition for operation.

---

10. Limitations

10.1 The evaluated build differs from the specified system. The build evaluated in §9 is the system of §7 run in process on one workstation. It departs from the production release candidate of §8 in where it runs, and in two checks.

Serving and storage. The harnesses of §9 call the fold, Sigma and the replay verifier in process, over event lists signed with keys the harness generates and a registry it holds in memory; no harness crosses a service boundary, reads a store or signs in. No figure of §9 therefore passes through the hosted configuration of §8: the ASGI services under their OpenAPI 3.1 contracts, the PostgreSQL event store with its transactional outbox and workers, the S3-compatible object store, the OIDC and OAuth client-credentials realms, the managed key service, the container images on infrastructure defined as code, and OpenTelemetry. That configuration is built, and its own acceptance measures it — eight families over 268 test cases, taken where PostgreSQL and the object store are reachable, among them a root replayed into either store reading every read model as its source does. The order invariance of §9.1 is a property of the fold, independent of the store, and the hosted store re-establishes it under its own sequence assignment and head signing. The key versions of §6.1 are held by the tests of §6.12; no Case of the corpus rotates a key.

Receipt signatures. Evidence receipt signature verification is partially stubbed. The Capsule carries signature references and the digest chain is computed and checked in full, so G3 holds for alteration after packaging. Sigma admits an external-evidence envelope on a verification flag the Capsule carries and checks no receipt signature itself. The deployment's intake checks a receipt signed in Ed25519 against the key the participant registry binds to the receipt's source identity, and refuses the receipt when the check fails; a receipt whose signature reference takes another form is admitted on the reference. Where the deployment reads a provider's Amazon CloudWatch under a role the provider granted, AWS signs nothing it returns. The deployment's collector signs what it received, from which endpoint and when, with a collection key the deployment's key service holds, and keeps the receipt outside the Case until the provider's administrator countersigns it with the organization's party key or the administrator's own member key. The countersigning key becomes the receipt's source identity, and only then does the receipt reach the intake and the Capsule. The harnesses of §9 file no receipt through the intake, so, in the corpus, attribution of a series to a party rests on the flag, which bounds the non-repudiation claim at the source. Case events are verified in full against the participant registry.

In the portfolio and divergence conditions, whose outcome turns on which party filed a series, the flag attributes each series.

Evidence-fault allocation. The Profile carries no term allocating the cost of an evidence fault — the term is a field of the contract's SQCD-1 addendum, checked when the addendum is authored — and arming does not check for a hard-required evidence class held by the party its absence benefits (§7.5). This bears on no figure in §9.

10.2 Binding the standard. Evaluation requires the input to name the standard it expects. The asserted digest commits to all four artifacts together with profile and version, and an evaluation that names none is refused. An input may additionally carry the artifacts as values, in which case resolution reads no store at all and the carried set is admitted only when it reproduces the asserted commitment, with no fallback to disk for a set that fails. A bundle carrying its Profile therefore replays to the same Sigma result, reason code and replay digest on a machine that holds no copy of the deployment's registry.

Two residues remain. A deployment must retain the artifacts a bundle references for as long as the bundle may be replayed, an operational obligation. And Corollary 5.1 binds the standard to a published head through the arming event; the strength of that binding is the strength of Theorem 5, which is one-sided and coarse at the resolution of the head publication interval, as §10.7 says of temporal binding generally.

10.3 The prices and the settlement terms are ours. Every priced figure in §9.3 and §9.4 is computed under commercial terms we wrote: a credit schedule, a unit price, a cap, a floor, and a shortfall basis. The system's commercial specification names the inputs to a remedy calculation — contract rules, invoice bindings, shortfall, exclusions, caps, floors, eligibility windows — and leaves the function that combines them to the contract. Our terms state it. The compute Profile's terms price shortfall as the weighted aggregate of per-channel costs, excluding the cross-source, spot-check and uncertainty terms, since a credit prices the delivery rather than the quality of the record about it. The terms file states the basis, and every quote carries the file's digest, so either party recomputes each price from the terms both pinned. A different basis would move every figure in §9.3 and §9.4, though not their direction, since the spread between the two collapses is positive under any monotone schedule. The priced corpus is a distribution over our generator under one price list, and the worked example is four deliveries under one Profile. Both illustrate magnitude, and neither measures real deliveries.

The settlement terms of §9.3 — the review window and the two forms — are also ours. Which form a counterparty should select, and whether a given jurisdiction will hold a party to the cost of silence it signed in advance, are questions for counsel, and the implementation's go-live checklist does not pass until counsel has reviewed them.

10.4 No production traffic. PSYCHE has not processed a delivery claim between real parties. The corpus in §9 is synthetic. That is the dominant threat to validity, and it falls on the claims we care most about.

We have not established that real contract language compiles into a Contract Profile without loss of the terms that decide disputes. We have not established that provider telemetry and customer scheduler records reconcile against a shared measurement window as cleanly as our constructions assume; clock discipline in particular is a place where synthetic data is likely to flatter the design. We have not established that the four-valued output partitions real cases usefully. A majority routed to `requires-review` would leave the pipeline delivering a well-documented referral in place of a determinate result. The §9.3 divergence figures are properties of our generator's condition distribution, and a real distribution would move them. We have not established which expiry forms real parties will select, or that a reviewer holding a VerificationReport decides faster than a reviewer holding today's materials.

Each is an empirical claim about the world, and a local implementation cannot support any of

them. A corpus of closed disputes would supply what is missing, since a resolved dispute contains the contract clause, both parties' evidence, and the outcome the parties reached, and therefore constitutes a labeled instance.

10.5 Gossip is an assumption about behavior. Theorem 4 detects equivocation only if participants exchange heads. The Profile can oblige them to, and the obligation is enforceable in the sense that a participant who never presents a head can be refused, but a pair who both neglect gossip are unprotected and the protocol cannot tell them so. The guarantee is conditional on a behavior, and we have not studied what fraction of the guarantee survives realistic gossip patterns. Certificate Transparency has the same dependency.

10.6 The separation principle is argued. §5 is an argument rather than a theorem. §6.13 formalizes the *difference* between the designs, and it does not adjudicate which parameter setting is right. The legal claim rests on the observation that acceptance is an authorized act rather than a fact about the world, which we take to be uncontroversial and which we have not tested against how a particular jurisdiction would treat a protocol-generated settlement decision. The commercial claim, that finance and procurement functions will not delegate acceptance authority to a third-party algorithm, rests on the structure of existing remedy processes and on analogy to adjacent markets. It has not been surveyed. Should a material population of enterprise buyers prove willing to bind themselves to an automated settlement decision, the strongest argument for this design weakens considerably. The four-finality decomposition and the formal model would survive.

10.7 Temporal binding is coarse and one-sided. Theorem 5 bounds creation from above at the resolution of head publication. It cannot establish that a record was created *early*, only that it was created before a head. A party that prepares a favorable record early and commits it before the window closes is indistinguishable from a party that recorded honestly. That outcome is correct, since §5.1 asks only for commitment under uncertainty. A party that could predict the outcome before the head defeats the argument, and nothing here prevents that. Every window of §5.7 inherits the same resolution: a window closes on the first readable head at or after its close, and no sooner.

10.8 The fabric's clock is trusted for wall time unless receipts corroborate it. §6.8 states the residue and the mitigation. Under a Profile requiring receipts, the fold reads no head time a quorum of observers has not corroborated, and it reports every receipt that contradicts its head. Under a Profile that requires none, every time-dependent guarantee in §6 — evidence commitment, standard binding, a grant's window, every window of §5.7 — rests on a value the fabric wrote. The transparency construction removes the fabric's ability to fork the record, and without receipts it leaves the fabric able to misreport when the record was written.

Receipts also arrive over time. Under a witnessed Profile, a receipt that arrives after a Case settled can make readable a head an event referenced, and with it admit an event the fold had refused, so two folds over the same log and heads differ until the parties exchange receipts. Committing receipts to the log before the events that rely on them would make the fold a function of the log and its heads alone (§11).

10.9 A divergence reaches no figure. When the parties' filings diverge on a hard-required channel, the Case returns `requires-review` (§7.3) and the engine reads no level for that chan-

nel. Declining to pick a figure keeps the allocation out of the verifier. It also lets a counterparty keep any Claim from a supported result by filing a diverging series; the Case then waits for its signature or expires under the form it signed. The series is signed and committed as it was measured, so the move is attributable, and the record states both figures; it does not say which is right. The settlement standard offers five other resolutions of a divergence, from no resolution to a tolerance mean, a third reading, a referee and a panel, each reaching a quantity before the party with standing decides [29, Annex 2.7]; the implementation carries only the sixth, carried to decision, and §11 lists the rest as open.

10.10 The participant registry is trusted. Every fold reads keys, roles and classes from the participant registry R it is given, and nothing on the Case log commits to that registry. Theorem 1 is a statement relative to R: it proves that the accepting signature verifies under the key R holds for the counterparty, and a fold given a registry that binds that key to someone else proves acceptance by someone else. A fold is as good as its registry, and a bundle's registry is as good as its source. The versioning of §6.1 fixes which key was in force when an event was sequenced and keeps every version; it does not put the registry on the log. Committing a registry digest in the arming would bind the parties to one registry for the Case, at the cost of a signed event for every key that rotates during it, and no Case of the §9 corpus rotates a key (§10.1).

Each organization holds its party key and its members' keys, in its administrators' and members' browsers or in its own key service; the network's keys (the fabric's, the verifier's, the quoter's and Sphinx's) are generated and held in the deployment's key service. An organization whose reviews are signed automatically may have the deployment host a signing delegate: a key in that same key service, registered as the organization's machine participant and bounded by the organization's grant within K. For what that delegate signs inside its grant, the organization trusts the deployment, and no proof of §6.6 reaches those signatures. The collection key that attests what the deployment's collector read from an organization's CloudWatch sits in the same key service, and a collected receipt becomes the organization's filing only under the organization's own countersignature.

---

11. Open Problems

Zero-knowledge Profile evaluation. §7.2 concedes that verifiers learn thresholds. The setting is commit-and-prove: commit to the full Profile, reveal only the predicate under evaluation, and prove that evaluation ran against the commitment. Whether the Profile's predicate language admits a practical proof system, and at what cost per Case, is open. A partial result — hiding thresholds while revealing structure — may be more tractable and would address most of the competitive exposure.

Pricing silence. §5.7 lets the parties price silence; it does not say what the price should be. EX-02 and EX-03 are symmetric positions, and which a counterparty should sign, and how a stated cost under EX-04 should scale with the value at risk and the cost of review, are open.

Delegated acceptance beyond a single Case. §5.6 and §6.1 fix the constitution and the per-Case grant: value ceiling, admitted Sigma results, revocation fixed by position in the authority chain, and a review-admission predicate whose failure is inert. Two extensions are not covered, and each requires enlarging the model itself.

A budget spanning Cases — a monthly ceiling across twenty Cases rather than a cap on one — makes grant state an object no per-Case fold can hold, since `F : L → S` is defined over one Case's log. It requires either a second fold over a log of grants or a redefinition of the subject, and Theorems 1 through 3 would need restating over the new subject.

Re-delegation of depth greater than one makes the principal harder to name, and §5.6 argues that authority is kept visible so that the principal can be named. A chain of grants is representable; a decision procedure establishing that a review at the end of a chain fell inside the authority at its head is not obviously cheap.

A Case no grant covers waits for the counterparty or expires. The protocol routes it nowhere else, because routing is a decision about who should decide, and the parties made that decision when they armed the Case: the counterparty decides, and its silence costs what the form says.

Censorship and late receipts. A fabric can decline to include an event and leave no proof (§6.6), and a receipt that arrives late can change what the fold reads (§10.8). Signed promises to include a submitted event within a stated delay would make the first attributable, and committing receipts to the log before the events that rely on them would close the second. Both change the event model, and Theorems 2b and 3 would need restating over it.

Resolving a divergence. §10.9 notes that the gate carries a divergence to decision and nothing else. Carrying the other forms — a tolerance mean, a third reading, a referee, a panel — as terms the parties select at arming, so that a divergence reaches a quantity before the party with standing decides, is open; each names a rule or a third party before the window, and the fold would have to read the quantity it produces without reading a decision into it.

Independent recomputation. Definition 1 is satisfied by construction in our implementation, so §9.1 tests our assembly rather than the format. Bit-identical recomputation by an independent implementation would strengthen the result, and disagreement between implementations would be more informative still, since it would locate an underspecification in the bundle.

Cross-layer netting. §2.3 describes obligations that fail to net across the facility, operator, and buyer layers. A Case-level record under pinned standards is a precondition for netting them. Whether netting is reachable in practice, given that the layers' contracts are separately negotiated and mutually confidential, is open. The privacy partition of §7.2 suggests a line of attack, since evaluable structure can cross a layer boundary where commercial terms cannot. A settled credit Case at one layer is a candidate piece of evidence at the next, and whether it can enter that layer's gate without its commercial terms is also open.

Standard drift. Profiles are pinned per window. Long-running contracts amend, and an amendment mid-window is handled crudely at present by requiring a new Profile and a window boundary. Whether obligations can be re-pinned without severing the Case spine, and what Theorem 3 should say about a Case whose standard changed beneath it, is open.

---

12. Conclusion

The market for contracted compute has outgrown the institutions that would ordinarily govern it. It reconciles bilaterally, under an exclusive-remedy structure that suppresses the disputes which would otherwise generate precedent, with no shared record. Something will

fill the gap.

The natural filling is to make obligations machine-checkable and to settle on the verification result. We have argued against the final step. Verification establishes evidentiary sufficiency, acceptance is an exercise of delegated authority, and a system deriving the second from the first produces a verdict that obliges the counterparty only where it authorized the verifier in advance, an authorization the proposals take when the obligation is bound, as policy, and this design records on the Case at review, bounded and revocable. §5.4 locates the defect precisely, and §6.13 states the same point in the model's terms, as a choice about who holds review authority.

PSYCHE implements the alternative. Its Sigma result is deliberately weaker: it asserts only that an evidence set supports a claim under a pinned standard, and it carries no power to settle. A Case reaches `accepted` only on a review signed by the party it binds, and its silence ends it in `expired`, at a cost both parties signed before the Case opened. Theorem 1 establishes that no other object can accept it, and Theorem 6 that no quantity of silence can. Against an untrusted fabric, Theorem 4 makes a forked history provable and attributable once participants compare the heads or proofs it lists, and Theorem 5 supplies the timing bound on which the evidentiary argument rests. We have shown that such a system can be built. Its invariants hold under adversarial logs, a party with no infrastructure access can recompute the derivation from a bundle, silence under settlement terms ends every Case in `expired` under the form its arming selected and accepts none, and the pipeline reduces a realistic delivery dispute to a single named object. We have not shown that it works on disputes between real parties, and §10.4 states what that would require.

Compute is now contracted in units of hundreds of millions of dollars and reconciled with none of the discipline §5.1 describes. A delivery record that meets that discipline — committed early, pinned to a standard both sides armed, and settled by the party that owes the money, at review, with the cost of its silence signed in advance — can be built today.

References

[1] Spheron. *GPU Cloud SLA Guarantees 2026: Uptime, Credits, Fine Print.* 2026. https://www.spheron.network/blog/gpu-cloud-sla-guarantees-2026/

[2] Data Center Dynamics. *The SLA mismatch at the center of the neocloud model.* 2026. https://www.datacenterdynamics.com/en/opinions/the-sla-mismatch-at-the-center-of-the-neocloud-model/

[3] A. de Valois-Franklin and A. Bogdan. *RAILS: Verification-Native Clearing for Agentic Commerce.* arXiv:2606.08790, June 2026.

[4] J. Rhodes and G. Kang. *Proof of Execution: Runtime Verification for Governed AI Agent Actions.* arXiv:2607.05397 [cs.CR], 2026.

[5] G. Zhu and C. Wang. *Explanation-Bound Tool Execution for AI Agents: Server-Verified Action Claims Without Trusting Model Rationales.* arXiv:2607.25364 [cs.AI], 2026.

[6] Compute Law Blog. *Cloud and hosting agreements and SLAs for AI workloads.* 2026. https://computelaw.blog/deals/cloud-hosting-agreements-slas-ai-workloads/

[7] B. Laurie, A. Langley, and E. Kasper. *Certificate Transparency.* RFC 6962, IETF, 2013.

[8] S. A. Crosby and D. S. Wallach. *Efficient Data Structures for Tamper-Evident Logging.* USENIX Security Symposium, 2009.

[9] A. Haeberlen, P. Kouznetsov, and P. Druschel. *PeerReview: Practical Accountability for Distributed Systems.* SOSP, 2007.

[10] N. Asokan, V. Shoup, and M. Waidner. *Optimistic Fair Exchange of Digital Signatures.* EUROCRYPT, 1998.

[11] H. Pagnia and F. C. Gärtner. *On the Impossibility of Fair Exchange without a Trusted Third Party.* Technical Report TUD-BS-1999-02, Darmstadt University of Technology, 1999.

[12] S. Haber and W. S. Stornetta. *How to Time-Stamp a Digital Document.* Journal of Cryptology, 3(2), 1991.

[13] C. Adams, P. Cain, D. Pinkas, and R. Zuccherato. *Internet X.509 Public Key Infrastructure Time-Stamp Protocol (TSP).* RFC 3161, IETF, 2001.

[14] M. Castro and B. Liskov. *Practical Byzantine Fault Tolerance.* OSDI, 1999.

[15] L. Moreau and P. Missier (eds.). *PROV-DM: The PROV Data Model.* W3C Recommendation, 2013.

[16] R. C. Merkle. *A Digital Signature Based on a Conventional Encryption Function.* CRYPTO, 1987.

[17] F. Zhang, E. Cecchetti, K. Croman, A. Juels, and E. Shi. *Town Crier: An Authenticated Data Feed for Smart Contracts.* ACM CCS, 2016.

[18] F. Zhang, D. Maram, H. Malvai, S. Goldfeder, and A. Juels. *DECO: Liberating Web Data Using Decentralized Oracles for TLS.* ACM CCS, 2020.

[19] M. Goenka, T. Pathak, and S. Asthana. *TessPay: Verify-then-Pay Infrastructure for Trusted Agentic Commerce.* arXiv:2602.00213 [cs.CR], 2026.

[20] F. Castillo, E. Brito, S. Werner, P. Pullonen-Raudvere, and J. Heiss. *Towards Trusted Service Monitoring: Verifiable Service Level Agreements.* International Conference on Service-Oriented Computing (ICSOC), 2025. arXiv:2510.13370 [cs.CR].

[21] F. Alpay and T. Alpay. *Unprivileged Topology Certificates for Cloud GPU Attestation.* arXiv:2606.24934 [cs.CR], 2026. Preprint; cited as an existence result for customer-side anchoring of accelerator identity, class and coarse location, not as a deployed evidence class.

[22] International Chamber of Commerce. *Uniform Customs and Practice for Documentary Credits*, 2007 revision (UCP 600), Article 34. Article text reproduced in *Abani Trading Pte Ltd v BNP Paribas* [2014] SGHC 111.

[23] *The Hague Rules as Amended by the Brussels Protocol 1968* (Hague-Visby Rules), Article III, rules 3, 4 and 5.

[24] *Brown Jenkinson & Co Ltd v Percy Dalton (London) Ltd* [1957] 2 QB 621 (CA).

[25] *In re Celsius Network LLC*, No. 22-10964 (Bankr. S.D.N.Y.); motion of 30 September 2022 concerning deployed hosting capacity and power cost pass-through. Resolved by the settlement of 15 September 2023 in *In re Core Scientific, Inc.*, No. 22-90341 (Bankr. S.D. Tex.), under which the site was transferred at an agreed value rather than the measurement adjudicated.

[26] *In re Rhodium Encore LLC*, No. 24-90448 (Bankr. S.D. Tex.), filed 24 August 2024; claims

concerning unpaid energy credits and outage-related losses, resolved by purchase and sale rather than by judgment.

[27] American Law Institute and Uniform Law Commission. *Uniform Commercial Code*, Article 2, § 2-606 (What Constitutes Acceptance of Goods). Text at https://www.law.cornell.edu/ucc/2/2-606

[28] H. Kalodner, S. Goldfeder, X. Chen, S. M. Weinberg, and E. W. Felten. *Arbitrum: Scalable, Private Smart Contracts.* USENIX Security Symposium, 2018.

[29] PSYCHE. *SQCD-1: Settlement Quality Compute Data.* Version 1.0, 3 September 2026. Section I.1.1 (silence), Section H (divergence), Annex 2.4 (expiry pricing) and Annex 2.7 (divergence resolution).